

CLAUDE CODE DEEP DIVE WORKSHOP

Chapter 1 - Overview

Agentic Coding의 새로운 표준, Claude Code 완전 정복

Update : 2026.09 / Claude Code v2.1 최신 채널 기준, 공식 문서 code.claude.com/docs 반영

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com

About this Workshop

3-Day Deep Dive into Claude Code

WORKSHOP MISSION

본 워크샵은 Claude Code를 실무에 즉시 적용 가능한 수준까지 학습합니다.

단순 사용법을 넘어 아키텍처, 보안, 자동화, SDK 통합까지 다룹니다.

DAY 1

Foundations

Overview, Subagents의
개념과 기본 사용법을 학습.
6시간 강의 + 2시간 실습.

DAY 2

Enterprise

Admin Setup, Settings로
대규모 조직 배포와 정책 구성
을 학습.
6시간 + 2시간.

DAY 3

Automation

CLI Reference, Agent
SDK로 자동화, 통합, 맞춤
에이전트 개발을 학습.
8시간.

OUTCOME

Production-Ready

워크샵 후 사내 환경에
Claude Code를 배포하고
자동화 파이프라인까지 구축.

3

Days of training

6

Major chapters

190+

Chapter 1 slides

15+

Hands-on labs

Agenda

Claude Code Deep Dive Workshop

- | | | |
|----|--------------------|--|
| 01 | Overview | Claude Code 정체성, 아키텍처, 설치, 인증, 핵심 기능 |
| 02 | Agents (Subagents) | 맞춤 서브에이전트 설계와 병렬 작업 위임 |
| 03 | Admin Setup | Enterprise 배포, 자격증명, 사내망 통합, 거버넌스 |
| 04 | Settings | settings.json, Permissions, Hooks, MCP 서버 통합 |
| 05 | CLI Reference | 전체 CLI 플래그, Headless 모드, CI/CD 자동화 |
| 06 | Agent SDK | Python, TypeScript SDK 기반 맞춤 에이전트 개발 |

Lab Environment

사전 준비사항 및 실습 환경

PREREQUISITES

아래 환경이 준비되어 있어야 모든 실습을 원활히 진행할 수 있습니다. 설치하는 챕터 1 Part 3에서 함께 진행합니다.

OS

macOS / Linux /
Windows

macOS 13+, Ubuntu
20.04+, Windows 10
1809+ 지원.
WSL2는 샌드박스까지 지원.

RUNTIME

Node.js 불필요

Native 설치가 표준이라
Node 불필요. npm 설치 경
로 사용 시에만 Node 18+
필요.

ACCOUNT

Claude / AWS / GCP

Claude 구독 또는
Console, AWS Bedrock
액세스 중 하나.
실습용 임시 자격증명 제공.

TOOLS

Git, IDE, Terminal

Git, VS Code 또는
JetBrains IDE, 익숙한
터미널 에뮬레이터를 준비
.

4GB+

RAM 최소 사양

x64/ARM64

지원 아키텍처

HTTPS 443

api.anthropic.com

ripgrep

설치에 기본 포함

Learning Outcomes

본 워크샵 종료 시 달성할 역량

BY THE END YOU WILL

Claude Code를 사용 가능한 수준이 아니라 운영 가능한 수준까지 학습합니다. 즉 도입, 보안, 운영, 자동화의 4축을 모두 다룹니다.

OUTCOME 1

Hands-on Mastery

디버깅, 리팩토링, 신기능 개발 등 일상 워크플로를 Claude Code로 실행.

OUTCOME 2

Customization

Subagents, CLAUDE.md, Hooks, MCP로 팀과 프로젝트에 맞춰 동작을 조정.

OUTCOME 3

Enterprise Ops

IAM 정책, 모델 통제, 사용량 모니터링으로 조직 단위 안전 배포.

OUTCOME 4

Automation

Headless 모드, Agent SDK로 CI/CD 통합과 자동화 파이프라인 구축.

4 Pillars

Mastery/Custom/Ops/Auto

15+ Labs

실습 중심 학습

Code

실제 코드와 명령 사용

Q&A

챗터별 토론 시간

01

Overview

Claude Code 정체성, 아키텍처, 설치부터 워크플로까지

CHAPTER 01 / PART 01

What is Claude Code

정체성, 모델, Agentic Coding 개념

1. 공식 정의와 Anthropic 배경, 최신 모델 라인업
2. Agentic Loop와 Code Completion 대비 차별점
3. 실행 환경 스펙트럼과 주요 사용 사례 6선
4. 구독 모델과 데이터 사용 정책

Chapter 1 Learning Objectives

본 챕터에서 달성할 학습 목표

AT THE END OF THIS CHAPTER

Claude Code가 무엇이고 왜 다른지 설명할 수 있으며, 본인 환경에 설치와 인증을 마치고 첫 세션을 운영할 수 있습니다.

OBJ 1

Define & Differentiate

Claude Code의 정의와
Agentic Coding 개념을 설명
하고 타 도구와 비교.

OBJ 2

Install & Authenticate

Native installer로 설치
6가지 인증 경로 중 환경에 맞는
것을 선택.

OBJ 3

Use Core Tools

Read, Edit, Bash, Grep 등
핵심 도구로 일상 코딩 작업
을 수행.

OBJ 4

Manage Context

CLAUDE.md, /clear,
/compact, Plan 모드로 세션
과 컨텍스트를 관리.

10 Parts

구조화된 학습 경로

5 Labs

Chapter 1 실습 랩

~3.5h

강의 시간

~1.5h

실습 시간

Chapter 1 Detailed Agenda

10 Parts / 195 Slides

PART 01	What is Claude Code	정체성, 모델, 사용 사례 / 26p
PART 02	Architecture	Agentic Harness, 도구, 컨텍스트 / 20p
PART 03	Installation	Native, Homebrew, WinGet, 패키지 / 20p
PART 04	Authentication	구독, Console, Bedrock, Gateway / 18p
PART 05	Quick Start	REPL, 슬래시 명령, 권한 모드 / 21p
PART 06	Core Capabilities	내장 도구 심화, Git, 권한 규칙 / 21p
PART 07	Interfaces	IDE, Desktop, Web, Slack, Chrome / 14p
PART 08	CLAUDE.md & Memory	메모리 계층, Rules, Auto memory / 17p
PART 09	Workflow Patterns	EPC, TDD, Headless, Routines / 18p
PART 10	Recap & Labs	요약, FAQ, 실습 5종, Q&A / 14p

Claude Code 공식 정의

터미널, IDE, 데스크톱, 브라우저에서 동작하는 Agentic Coding 도구

DEFINITION

Claude Code는 코드베이스를 읽고, 파일을 편집하고, 명령을 실행하며 개발 도구와 통합되는 Anthropic의 Agentic Coding 도구입니다. 자연어 지시를 받아 멀티스텝 작업을 자율적으로 완수합니다.

WHAT

Agentic Tool

자율적으로 도구를 호출하며 멀티스텝 작업을 완수하는 에이전트형 도구.

WHERE

Everywhere

터미널, VS Code, JetBrains, Desktop 앱, 웹, Slack까지 동일 엔진.

WHO BY

By Anthropic

Claude 모델을 만든 Anthropic의 1차 도구로 모델 통합이 가장 깊음.

CLI+

터미널 중심, 전 환경 확장

Sonnet 5

기본 주력 모델

2024

최초 출시

GA

정식 출시 상태

Anthropic 회사 배경

AI Safety 중심의 프론티어 모델 개발사

WHO IS ANTHROPIC

Anthropic은 안전하고 신뢰 가능한 AI를 목표로 하는 프론티어 AI 기업입니다.

Claude 모델 패밀리와 Claude Code, Agent SDK를 직접 개발합니다.

FOCUS

AI Safety First

Constitutional AI 등 안전 연구를 제품에 직접 반영하는 개발 철학.

MODELS

Claude Family

Fable, Opus, Sonnet, Haiku로 이어지는 프론티어 모델 라인업.

PRODUCTS

Code + Platform

Claude Code, Claude 앱, Agent SDK, Claude API를 통합 제공.

AWS

Bedrock Partner

AWS Bedrock에서 Claude 모델을 제공하는 전략적 파트너십.

2021

회사 설립

Fable 5.1

최신 최상위 모델

Bedrock

AWS 공식 제공

1P Tool

모델과 도구 동일 개발사

Claude 모델 패밀리

Claude Code가 활용하는 최신 모델 라인업 (2026.09)

Claude Fable 5.1	최상위 Mythos-class, 1M, 장시간 자율 세션	가장 어려운 대규모 작업
Claude Opus 5	깊은 추론, 복잡한 아키텍처 결정	복잡한 리팩토링, 설계
Claude Sonnet 5	기본 모델, 네이티브 1M 컨텍스트	일상 코딩 작업 전반
Claude Haiku 4.5	빠른 응답, 비용 효율	간단한 변환, 검색, 분류

1M	/model	v2.1.197+	v2.1.257+
Sonnet 5 컨텍스트	세션 중 전환 명령	Sonnet 5 요구 버전	Fable 5.1 요구 버전

모델 Alias 시스템

버전 번호 대신 별칭으로 모델 선택

default	계정 유형별 권장 모델로 복귀하는 특수 값	오버라이드 해제
best	Fable 5.1 접근 가능 시 Fable 5.1, 아니면 최신 Opus	최고 성능 자동 선택
fable / opus	Fable 5.1 또는 최신 Opus 지정	난제, 장기 세션
sonnet / haiku	최신 Sonnet 또는 Haiku 지정	일상 작업, 경량 작업
opus[1m]	1M 토큰 컨텍스트 윈도우로 Opus 사용	초장기 세션
opusplan	Plan 모드는 opus, 실행은 sonnet으로 자동 전환	계획 품질과 실행 비용 균형

Agentic AI 개념

Reactive에서 Agentic으로의 패러다임 전환

DEFINITION

Agentic AI는 사용자의 목표를 받아 스스로 계획을 세우고, 도구를 사용하며, 결과를 관찰하면서 목표 달성까지 자율적으로 작동하는 AI 시스템입니다.

PROPERTY 1

Goal-driven

단계별 명령이 아닌 최종 목표를 받아 스스로 경로를 결정.

PROPERTY 2

Tool-using

파일 읽기, 명령 실행, 웹 검색 등 외부 도구를 능동 호출.

PROPERTY 3

Observative

도구 결과를 관찰하고 다음 행동을 결정하는 피드백 루프.

PROPERTY 4

Iterative

실패하면 다른 접근을 시도하며 완료까지 반복, 자가 수정.

Gather

컨텍스트 수집

Act

도구로 행동

Verify

결과 검증

Repeat

완료까지 반복

Agentic Loop 3단계

Gather Context, Take Action, Verify Results

THE AGENTIC LOOP

작업을 주면 Claude는 세 단계를 섞어가며 수십 개의 행동을 연결합니다.

사용자는 언제든지 Esc로 개입해 방향을 조정할 수 있습니다.

STEP 01

Gather Context

파일 검색과 읽기로 코드 이해,
CLAUDE.md와 메모리 로드

STEP 02

Take Action

파일 편집, 명령 실행, 커밋 등 실제
변경 수행

STEP 03

Verify Results

테스트 실행, 타입 검사, 결과 관찰 후
다음 판단

LOOP

Repeat / Steer

완료까지 반복, Esc로 중단하거나
지시 추가 가능

Agentic Coding vs Code Completion

자동완성과 에이전트의 근본적 차이

KEY DIFFERENCE

자동완성은 커서 위치에서 다음 코드를 제안하고, Agentic Coding은 목표를 받아 저장소 전체에서 작업을 완수합니다.

CODE COMPLETION (기존)

1. 현재 파일, 커서 주변 컨텍스트
2. 다음 몇 줄을 제안
3. 실행과 검증은 개발자 몫
4. IDE 안에서만 동작
5. 예: 함수 시그니처 자동완성

AGENTIC CODING (Claude Code)

1. 저장소 전체 + 터미널 + git 상태
2. 여러 파일을 편집하고 명령 실행
3. 테스트로 스스로 검증 후 수정
4. 터미널, IDE, 웹 어디서든 동작
5. 예: 실패 테스트 원인 추적과 수정

주요 코딩 AI 도구 비교

포지셔닝 관점의 개괄 비교

Claude Code	터미널 중심 에이전트, 전 환경 확장, SDK 제공	저장소 단위 자율 작업
GitHub Copilot	IDE 자동완성 출발, 에이전트 기능 확장 중	IDE 내 코드 제안
Cursor	AI 네이티브 IDE, 에디터 경험 중심	IDE 통합 워크플로
OpenAI Codex	클라우드 에이전트와 CLI 병행	위임형 클라우드 작업
Gemini CLI	Google 생태계 CLI 에이전트	GCP 연계 작업

Harness	SDK	Bedrock	2026.09
모델 + 도구 통합 깊이	맞춤 에이전트 개발	엔터프라이즈 경로	비교 기준 시점

실행 환경 스펙트럼

동일 엔진, 모든 곳에서 (Use Claude Code Everywhere)

ENV 1

Terminal CLI

모든 기능의 기준점. 파이프와 스크립트 등 Unix 조합성.

ENV 2

VS Code / Cursor

인라인 diff, @-mention, 플랜 검토를 에디터에서 제공.

ENV 3

JetBrains

IntelliJ, PyCharm 등에서 대화형 diff와 컨텍스트 공유.

ENV 4

Desktop App

병렬 세션, 시각적 diff, 예약 작업, 클라우드 세션 실행.

ENV 5

Web / iOS

claude.ai/code에서 설치 없이 클라우드로 실행, 모바일 연속 작업.

ENV 6

Slack / CI / Chrome

@Claude 멘션, GitHub Actions, 라이브 웹앱 디버깅 통합.

왜 터미널 기반인가

CLI가 여전히 기준점인 이유

UNIX PHILOSOPHY

터미널에서 할 수 있는 모든 것을 Claude도 할 수 있습니다.

조합 가능하고, 파이프로 연결되며, 스크립트와 CI에 그대로 들어갑니다.

REASON 1

도구 접근성

빌드, git, 패키지 매니저 등 개발 도구가 모두 CLI에 존재.

REASON 2

조합 가능성

파이프, 리다이렉트, jq와 결합해 워크플로를 자유롭게 구성.

REASON 3

자동화 친화

-p 헤드리스 모드로 CI/CD와 스크립트에 즉시 통합.

REASON 4

IDE 종립

특정 에디터에 종속되지 않고 모든 개발 환경과 공존.

Pipe

stdin/stdout 연계

-p

헤드리스 플러그

Any IDE

에디터 비종속

CI-ready

파이프라인 즉시 통합

Claude Code의 4대 핵심 가치

Why teams adopt Claude Code

WHY IT MATTERS

이 네 가지 가치는 개인 생산성 향상을 넘어, 팀과 조직 단위로 코딩 방식을 재정의할 수 있는 기반을 제공합니다.

VALUE 1

깊은 코드 이해

단일 파일이 아닌 프로젝트 전체를 자율 탐색하며 컨텍스트 구축.

VALUE 2

실제 작업 완수

제안에 그치지 않고 파일 수정, 테스트 실행, PR 생성까지 수행.

VALUE 3

안전한 권한 모델

allow, ask, deny 규칙과 체크포인트로 동작 범위를 통제.

VALUE 4

확장 가능한 통합

Skills, Subagents, MCP, Hooks, SDK로 워크플로에 맞춰 확장.

Repo-wide

컨텍스트 범위

End-to-end

작업 완수 수준

Granular

권한 제어

SDK + MCP

확장성

자율성 스펙트럼

Plan에서 Auto까지, 권한 모드 4종

CONTROL DIAL

Shift+Tab으로 모드를 순환합니다. 오른쪽으로 갈수록 자율성이 커지고 개입이 줄어듭니다.

작업 위험도에 맞춰 선택합니다.

MODE 1

Plan

소스 수정 없이 탐색과 계획만
, 승인 후 실행

MODE 2

Default

파일 편집과 셸 명령 전 매번
질문

MODE 3

Accept Edits

편집과 mkdir 등 파일 작업
자동, 그 외 질문

MODE 4

Auto

백그라운드 안전 검사로 전 행
동 평가, 리서치 프리뷰

CI 등 비대화 환경은 --permission-mode 플래그와 사전 합의된 allow 규칙으로 운영

주요 사용 사례 6선

Claude Code가 가장 큰 가치를 발휘하는 시나리오

CASE 1

디버깅

에러 메시지에서 원인을 추적하고 수정안을 제시, 적용.

CASE 2

리팩토링

대규모 코드베이스의 일관된 패턴 변경과 구조 개선.

CASE 3

신기능 개발

요구사항으로부터 계획, 구현, 테스트까지 일괄 처리.

CASE 4

코드베이스 학습

낯선 프로젝트의 구조와 흐름을 자연어 대화로 학습.

CASE 5

테스트 작성

커버리지 갭 분석과 자동 테스트 생성, 실패 수정.

CASE 6

DevOps 자동화

헬 스크립트, IaC, CI/CD 파이프라인 작성과 점검.

Case 1 / 디버깅

에러에서 원인 추적, 수정, 검증까지

● ● ● TERMINAL SESSION

```
$ npm test
# FAIL src/auth/session.test.ts
#   TypeError: Cannot read properties of undefined
```

```
$ claude "npm test가 실패해. 원인을 찾아 고치고
테스트가 통과하는지 확인해줘"
```

```
# Claude의 진행 순서
# 1. 테스트 실행해 실패 로그 확보
# 2. 스택트레이스의 session.ts 읽기
# 3. null 가드 누락 확인 후 Edit
# 4. npm test 재실행으로 통과 검증
```

WHY IT WORKS

- 에러 메시지가 곧 시작 컨텍스트
- 스스로 테스트를 돌려 검증
- 여러 파일에 걸친 원인도 추적
- 수정 전 스냅샷으로 되돌리기 가능

Case 2 / 리팩토링

저장소 전반의 일관된 패턴 변경

● ● ● TERMINAL SESSION

```
$ claude
> src/ 전체에서 콜백 스타일 fs 호출을 fs/promises 기반 async/await로 바꿔줘. 변경 후 lint와 테스트로 검증해줘.
```

```
# Claude의 진행 순서
# 1. Grep으로 fs.readFile( 등 호출 지점 수집
# 2. 파일별로 Edit, import 구문까지 일괄 정리
# 3. npm run lint && npm test 실행
# 4. 실패 지점 재수정 후 요약 보고
```

SCALE ADVANTAGE

- 수십 개 파일도 동일 규칙 적용
- import 정리 등 부수 변경 포함
- 린트와 테스트로 회귀 방지
- /diff로 전체 변경 한눈에 검토

Case 3 / 신기능 개발

요구사항에서 계획, 구현, 테스트까지

● ● ● TERMINAL SESSION (PLAN MODE)

```
$ claude
```

```
> Shift+Tab 두 번으로 Plan 모드 진입
```

```
> /users API에 커서 기반 페이지네이션을 추가하고 싶어. 먼저 계획을 세워줘.
```

```
# Plan 모드 산출물 (예시)
```

```
# 1. 스키마: cursor, limit 파라미터 정의
```

```
# 2. 변경 파일: routes, service, repo 3곳
```

```
# 3. 테스트: 경계값 4케이스 추가
```

```
> 계획 승인. 구현하고 테스트까지 실행해줘.
```

TWO-PHASE FLOW

- 탐색과 계획을 코드와 분리
- 계획 검토 후 방향 확정
- 구현은 승인된 계획을 따라 진행
- opusplan alias로 품질과 비용 균형

Case 4 / 코드베이스 학습

낮선 프로젝트를 대화로 파악

● ● ● TERMINAL SESSION

```
$ cd unfamiliar-project && claude
```

- > 이 프로젝트의 전체 구조를 설명해줘.
- > 인증은 어디서 어떻게 처리돼?
- > 결제 실패 시 재시도 로직을 따라가며 호출 흐름을 정리해줘.

```
# Claude의 동작
```

```
# Glob, Grep, Read로 구조 탐색
```

```
# 진입점부터 호출 체인 추적
```

```
# 다이어그램 수준의 텍스트 요약 제공
```

ONBOARDING VALUE

- 온보딩 기간을 며칠에서 시간 단위로
- 질문할수록 깊어지는 이해
- 문서화되지 않은 흐름도 추적
- 학습 결과를 CLAUDE.md에 축적 가능

Case 5 / 테스트 작성

커버리지 갭 분석과 자동 생성

● ● ● TERMINAL SESSION

```
$ claude "auth 모듈의 테스트를 작성하고 실행해서 실패하면 수정까지 해줘"
```

```
# Claude의 진행 순서
```

- # 1. 기존 테스트와 커버리지 현황 파악
- # 2. 미커버 분기 정리 (만료 토큰 등)
- # 3. 프로젝트 컨벤션에 맞춰 테스트 작성
- # 4. 실행, 실패 시 원인 수정 후 재실행

```
$ claude -p "커버리지 80% 미만 파일 목록" \  
--allowed-tools "Bash(npm run coverage),Read"
```

QUALITY LOOP

- 작성과 실행, 수정을 한 번에
- 엣지 케이스를 능동적으로 발굴
- 프로젝트 테스트 컨벤션 준수
- 헤드리스로 커버리지 리포트 자동화

Case 6 / DevOps 자동화

스크립트, IaC, 파이프라인 작성과 점검

● ● ● TERMINAL SESSION

```
$ claude "이 Dockerfile을 멀티스테이지로  
최적화하고 이미지 크기 변화를 보고해줘"  
  
$ claude "terraform plan 결과를 읽고  
위험한 변경이 있는지 요약해줘"  
  
$ tail -200 app.log | claude -p \  
"이상 징후가 있으면 원인 후보와 함께 보고"  
  
# git, docker, terraform, kubectl 등  
# 터미널의 모든 CLI 도구를 그대로 활용
```

OPS SURFACE

- 로그 파이프 분석이 한 줄로
- IaC 변경 리뷰의 1차 필터
- CI 실패 원인 자동 진단
- 터미널 도구 생태계 전체가 무기

라이선스 및 구독 모델

개인부터 엔터프라이즈까지

Claude Pro / Max	월 정액 구독, OAuth 로그인, 개인 사용량	개인 개발자, 파워 유저
Team / Enterprise	중앙 청구, SSO, 정책 통제, 관리 콘솔	조직 단위 도입
Claude Console	API 종량제, Claude Code 전용 롤 지원	API 청구 선호 조직
Bedrock / Vertex / Foundry	클라우드 자격증명, 자체 클라우드 청구	엔터프라이즈 표준 경로
Free plan	Claude Code 미포함	웹 채팅 전용

OAuth
구독 로그인 방식

SSO
Enterprise 지원

Bedrock
AWS 청구 통합

No Free
무료 플랜은 미지원

데이터 보안 및 프라이버시

코드가 어디로 가고 어떻게 보호되는가

TRUST BOUNDARY

로컬 실행 시 코드는 사용자의 머신에 있고, 모델 추론에 필요한 컨텍스트만 API로 전송됩니다. 전송 구간과 저장 구간 모두 암호화됩니다.

LOCAL

로컬 우선 실행

파일 접근과 명령 실행은 사용자 머신에서, 권한 규칙으로 통제.

TRANSIT

TLS 전송 암호화

API 통신은 TLS로 보호, 사내 프록시와 커스텀 CA도 지원.

CLOUD

클라우드 경로 선택

Bedrock 사용 시 트래픽이 AWS 계정 경계 안에서 처리.

CONTROL

조직 통제 수단

managed settings, 감사 로그, ZDR 옵션으로 정책 강제.

TLS

전송 구간 암호화

AES-256

저장 데이터 암호화

ZDR

Zero Data Retention 옵션

VPC

Bedrock PrivateLink 경로

Anthropic 데이터 사용 정책 핵심

한 페이지 요약

학습 사용

상업용 계정 기본 미사용, 명시 동의 시만

Console, Bedrock 경로 포함

보존 기간

안전성 검토 목적 제한적 보존

ZDR 계약 시 보존 없음

인간 검토

약관 위반 의심 등 예외 상황에 한정

자동화 시스템 우선

제3자 공유

법적 의무 외 없음

소환장 등 법적 요구 한정

암호화

TLS in transit, AES-256 at rest

모든 경로 동일

Opt 기반

학습 사용 통제

ZDR

보존 제로 옵션

Bedrock

AWS 데이터 경계

docs

data-usage 문서 확인

Part 1 Recap

What is Claude Code 핵심 정리

KEY TAKEAWAY

Claude Code는 Agentic Loop로 저장소 단위 작업을 완수하는 Anthropic 1차 도구이며, 이제 터미널을 넘어 모든 개발 표면에서 동일 엔진으로 동작합니다.

핵심 정리

1. 정의: 읽고, 편집하고, 실행하는 에이전트
2. 루프: Gather, Act, Verify 반복
3. 모델: Fable 5.1, Opus 5, Sonnet 5, Haiku
4. 환경: CLI, IDE, Desktop, Web, Slack
5. 가치: 이해, 완수, 통제, 확장

다음 파트

1. Part 2 Architecture로 이동
2. Agentic Harness 구조 분해
3. 도구 시스템과 컨텍스트 관리
4. 인증 우선순위와 모델 라우팅
5. 체크포인트와 보안 아키텍처

CHAPTER 01 / PART 02

Architecture

Agentic Harness의 내부 구조

1. 인터페이스, 하네스, 모델, 공급자 4계층
2. 도구 시스템과 권한 모델
3. 컨텍스트 윈도우 관리와 Compaction
4. 체크포인트, 샌드박스, 텔레메트리

전체 아키텍처 / Agentic Harness

언어 모델을 코딩 에이전트로 만드는 4계층

AGENTIC HARNESS

Claude Code는 모델 주위의 하네스입니다. 도구, 컨텍스트 관리, 실행 환경을 제공해 언어 모델을 유능한 코딩 에이전트로 바꿉니다.

Interface

Terminal CLI / VS Code / JetBrains / Desktop / Web / Slack / CI, 동일 엔진의 표면

Harness

도구 실행, 권한 검사, 컨텍스트와 메모리 관리, 체크포인트, 세션 저장

Model

Fable 5.1, Opus 5, Sonnet 5, Haiku 4.5, alias로 선택하고 /model로 전환

Provider

Claude API / Amazon Bedrock / Google Vertex AI / Microsoft Foundry / Gateway

실행 환경 3종

코드가 어디서 실행되는가

EXECUTION ENVIRONMENTS

인터페이스와 별개로, 코드가 실제 실행되는 위치는 세 가지입니다.

위험 모델과 작업 성격에 맞춰 선택합니다.

ENV 1

Local

기본값. 내 머신에서 실행, 파일과 도구,
환경에 완전 접근.

ENV 2

Cloud

Anthropic 관리 VM에서 실행.
작업 오프로드, 원격 저장소 작업.

ENV 3

Remote Control

실행은 내 머신, 조작은 브라우저.
로컬 유지 + 웹 UI.

Local

기본 실행 위치

VM

클라우드 세션 격리

--teleport

웹 세션을 터미널로

--remote

터미널에서 클라우드로

Client Layer 상세

인터페이스별 특성과 최적 용도

Terminal CLI	전체 기능 기준점, 파이프와 스크립트	일상 작업, 자동화
VS Code / Cursor	인라인 diff, @-mention, 플랜 검토	에디터 중심 개발
JetBrains	대화형 diff, 선택 영역 컨텍스트	IntelliJ 계열 사용자
Desktop App	병렬 세션, 예약 작업, Dispatch	멀티 세션 운영
Web / iOS	클라우드 세션, 설치 불필요	원격, 이동 중 작업
Slack / CI	@Claude 멘션, Actions 통합	팀 협업, 파이프라인

Agent SDK의 위치

같은 하네스를 라이브러리로

SDK POSITION

Agent SDK는 Claude Code의 하네스를 Python과 TypeScript 라이브러리로 노출합니다.

CLI가 쓰는 것과 동일한 도구와 권한 체계를 앱에 내장할 수 있습니다.

CLAUDE CODE CLI

1. 대화형 REPL과 헤드리스 모드
2. 사람이 주도하는 워크플로
3. 설정은 settings.json 계층
4. 즉시 사용, 설치만 하면 동작

AGENT SDK

1. query() 등 프로그래밍 인터페이스
2. 앱과 서비스에 에이전트 내장
3. 도구, 권한, 혹은 코드로 제어
4. 챕터 6에서 심화 학습

API 통신 경로

하네스에서 공급자까지

PROVIDER ROUTING

하네스는 설정된 공급자 하나로 추론 요청을 보냅니다.

환경변수 스위치로 경로를 선택하고, 조직은 게이트웨이로 중앙 집중할 수 있습니다.

SOURCE

Claude Code

프롬프트와 도구 결과를 요청
으로 구성

PATH 1

Claude API

기본 경로,
api.anthropic.com

PATH 2

Bedrock / Vertex /
Foundry

USE_BEDROCK 등 스위치로
선택

PATH 3

LLM Gateway

ANTHROPIC_BASE_URL,
조직 중앙 관문

인증 우선순위 6단계

여러 자격증명이 공존할 때의 선택 순서

1 Cloud Provider	CLAUDE_CODE_USE_BEDROCK / VERTEX / FOUNDRY 설정 시 최우선
2 AUTH_TOKEN	ANTHROPIC_AUTH_TOKEN, Bearer 헤더로 전송, 게이트웨이 인증용
3 API_KEY	ANTHROPIC_API_KEY, X-API-Key 헤더, 대화형에서는 1회 승인 필요
4 apiKeyHelper	설정 스크립트가 반환하는 동적 키, 볼트 연동과 회전에 적합
5 OAUTH_TOKEN	claude setup-token으로 만든 1년 토큰, CI 파이프라인용
6 /login OAuth	구독 사용자 기본값, Pro / Max / Team / Enterprise

Model Routing 상세

공급자별 alias 해석 (2026.09)

Claude API

opus는 Opus 4.8, sonnet은 Sonnet 5

최신 모델 최우선 제공

Claude Platform on AWS

opus는 Opus 4.7, sonnet은 Sonnet 4.6

AWS 인증 + 1P API

Bedrock / Vertex / Foundry

opus는 Opus 4.6, sonnet은 Sonnet 4.5

전체 모델명으로 상위 지정 가능

오버라이드

ANTHROPIC_DEFAULT_OPUS_MODEL 등 환경변수

alias가 가리킬 버전 고정

조직 통제

availableModels 관리 설정으로 허용 목록

picker와 플래그 전부 강제

Tool System 개요

다섯 가지 도구 카테고리

TOOLS MAKE IT AGENTIC

도구가 없으면 모델은 텍스트만 반환합니다.

도구가 있어야 읽고, 고치고, 실행하고, 검색하며 루프를 돌 수 있습니다.

CAT 1

File Ops

Read, Edit, Write,
NotebookEdit

CAT 2

Search

Glob, Grep, 코드베
이스 탐색

CAT 3

Execution

Bash,
PowerShell, git,
테스트

CAT 4

Web

WebSearch,
WebFetch,
문서 조회

CAT 5

Code Intel

LSP:
타입 오류, 정의 이동

내장 도구 카탈로그

Tools Reference 기준 주요 도구

Read / Edit / Write

파일 읽기, 정밀 수정, 신규 생성

Edit, Write는 승인 필요

Glob / Grep

패턴 파일 찾기, 내용 정규식 검색

승인 불필요

Bash / PowerShell

셸 명령 실행, Windows 네이티브 지원

승인 필요

WebSearch / WebFetch

웹 검색과 URL 콘텐츠 조회

도메인 규칙 적용 가능

Agent / SendMessage

서브에이전트 생성, 에이전트 간 메시지

챕터 2에서 심화

LSP / Monitor / Cron

코드 인텔리전스, 로그 감시, 예약

최근 추가된 도구

신규 도구 하이라이트

2026년 상반기에 추가된 능력

NEW 1

LSP

편집 직후 타입 오류 확인,
정의 이동, 참조 찾기.

NEW 2

Monitor

백그라운드 명령 출력을
실시간 관찰하고 반응.

NEW 3

AskUserQuestion

요구사항이 모호할 때
객관식으로 사용자에게 질문.

NEW 4

Artifact

세션 결과를 조직 내 공유
가능한 라이브 페이지로
발행.

Cron 3종

세션 내 예약 도구

RemoteTrigger

Routines 백엔드

EnterWorktree

격리 워크트리 진입

ReportFindings

리뷰 결과 구조화

MCP Tool Integration

외부 서비스를 도구로 연결

MODEL CONTEXT PROTOCOL

MCP는 AI 도구를 외부 데이터 소스에 연결하는 개방형 표준입니다.

GitHub, Slack, 사내 시스템의 기능이 Claude의 도구로 등록됩니다.

CONNECT

서버 연결

claude mcp add로 stdio,
HTTP 서버를 등록하고
/mcp로 상태 확인.

SCALE

Tool Search

도구 정의는 기본 지연 로딩,
사용 시점에 온디맨드 로딩
으로 컨텍스트 절약.

AUTH

인증

claude mcp login으로 셸
에서 OAuth 인증, 조직은
허용 목록 통제.

GOVERN

관리

managed 설정으로 서버
allowlist, denylist를 조직
차원에서 강제.

stdio/HTTP

전송 방식

Deferred

도구 정의 지연 로딩

/mcp

서버별 비용 확인

Ch.4

MCP 심화 챕터

권한 모델

규칙과 모드의 이중 구조

TWO LAYERS OF CONTROL

권한은 도구별 규칙과 세션 모드의 조합으로 결정됩니다.

규칙이 정적 정책이라면 모드는 세션의 자율성 다이얼입니다.

RULES (settings.json)

1. allow: 묻지 않고 실행
2. ask: 실행 전 확인
3. deny: 무조건 차단
4. Bash(npm test) 등 도구별 패턴
5. 조직 managed 설정이 최우선

MODES (Shift+Tab)

1. Plan: 소스 수정 없이 계획
2. Default: 편집과 명령마다 확인
3. Accept Edits: 파일 작업 자동
4. Auto: 백그라운드 안전 검사
5. 비대화는 --permission-mode

Context Window 구성

세션 시작 시 무엇이 로드되는가

CONTEXT LAYOUT (/context 로 확인)

```
# 세션 컨텍스트에 들어가는 것들
System instructions      하네스 기본 지침
CLAUDE.md (계층 병합)   프로젝트, 사용자, 조직 지침
Auto memory (MEMORY.md) 첫 200줄 또는 25KB
Skills 설명부           본문은 사용 시점 로드
MCP 도구 이름           정의는 온디맨드 로드
대화 이력 + 도구 출력   세션이 길수록 증가

$ /context      # 사용량을 색상 그리드로 시각화
$ /mcp          # 서버별 컨텍스트 비용 확인
```

/context

구성 시각화 명령

200줄/25KB

Auto memory 로드 상한

On-demand

Skills, MCP 본문

1M

Sonnet 5 윈도우

Context Compaction 상세

한계에 다가갈 때의 자동 처리

AUTO COMPACTION

한계에 접근하면 오래된 도구 출력부터 정리하고, 필요하면 대화를 요약합니다.

초반의 세부 지시는 사라질 수 있으니 영속 규칙은 CLAUDE.md에 둡니다.

STAGE 1

임계 감지

윈도우 사용량이 한계에 접근

STAGE 2

출력 정리

오래된 도구 출력부터 제거

STAGE 3

대화 요약

요청과 핵심 코드는 보존

GUARD

Thrashing 방지

반복 실패 시 루프 대신 오류 표시

Checkpointing

모든 파일 편집은 되돌릴 수 있다

EVERY EDIT IS REVERSIBLE

Claude가 파일을 편집하기 전에 해당 파일의 현재 내용을 스냅샷으로 남깁니다.

Esc 두 번이나 **/rewind** 로 이전 상태로 되돌립니다.

SNAPSHOT

편집 전 저장

영향받는 파일을 자동 스냅샷
, git과는 별개의 세션 로컬
기록.

REWIND

Esc Esc / /rewind

코드와 대화를 특정 시점으로
롤백, /clear 이전으로도 복
귀.

SCOPE

파일 변경 한정

DB, API, 배포 등 외부 부작용은
미포함, 그래서 실행 전 확인.

STORAGE

JSONL 세션 기록

~/.claude/projects/ 아래
평문 기록으로 resume와
fork 지원.

Esc Esc

빠른 되감기

/rewind

체크포인트 선택

Local

세션 로컬, git 별개

Files only

외부 부작용 제외

Security Architecture

네 겹의 방어선

LAYER 1

Permissions

allow, ask, deny 규칙과
4가지 모드로 행동 사전
통제.

LAYER 2

Sandboxing

샌드박스 Bash로 파일시스
템과 네트워크 격리 실행.

LAYER 3

Checkpoints

편집 전 스냅샷으로 사후
복구 경로 확보.

LAYER 4

Audit

세션 기록, Hooks, OTel로
행위 추적과 정책 검증.

4 Modes

권한 모드

FS + Net

샌드박스 격리 축

Snapshot

편집 전 자동 저장

OTel

표준 텔레메트리

네트워크 트래픽 패턴

방화벽 규칙 설계 기준

api.anthropic.com

모델 추론 요청, HTTPS 443

기본 경로

claude.ai / downloads

로그인 콜백, 설치와 업데이트

설치 채널

bedrock-
runtime.*.amazonaws.com

Bedrock 경로 추론

PrivateLink 가능

MCP 서버 endpoint

등록한 서버별 상이

조직 allowlist 권장

statsig / sentry 계열

기능 플래그, 오류 보고

환경변수로 비활성 가능

Logging & Telemetry

로컬 기록과 OpenTelemetry

OBSERVABILITY

세션은 로컬 JSONL로 기록되고, 조직 모니터링은 OpenTelemetry 표준으로 내보냅니다.

LOCAL RECORDS

1. ~/.claude/projects/ 세션 JSONL
2. resume, fork, rewind의 기반
3. claude --debug 상세 로그
4. /export로 대화 내보내기

ORG TELEMETRY (OTel)

1. 토큰, 비용, 도구 사용 메트릭
2. OTLP로 CloudWatch, Datadog 연계
3. 사용자, 팀 단위 대시보드
4. 설정 방법은 챕터 3에서 실습

Part 2 Recap

Architecture 핵심 정리

KEY TAKEAWAY

Claude Code는 모델을 감싼 하네스입니다.

도구와 권한, 컨텍스트 관리, 체크포인트가 하네스의 본체이며, 공급자 선택이 데이터 경계를 결정합니다.

핵심 정리

1. 4계층: Interface, Harness, Model, Provider
2. 실행 위치: Local, Cloud, Remote Control
3. 인증 우선순위 6단계 암기
4. 도구 5 카테고리 + MCP 확장
5. Compaction과 Checkpoint 동작

다음 파트

1. Part 3 Installation으로 이동
2. Native 설치가 새 표준
3. Homebrew, WinGet, 리눅스 저장소
4. 릴리스 채널과 버전 통제
5. Docker와 Devcontainer

CHAPTER 01 / PART 03

Installation

Native가 새 표준, 채널과 버전 통제까지

1. 시스템 요구사항과 설치 방법 5종 비교
2. Native install: macOS, Linux, WSL, Windows
3. Homebrew, WinGet, apt, dnf, apk 저장소
4. 릴리스 채널, 버전 고정, 검증과 제거

시스템 요구사항

지원 플랫폼과 하드웨어 (2026.09)

OS	macOS 13+, Windows 10 1809+, Server 2019+	Ubuntu 20.04+, Debian 10+, Alpine 3.19+
하드웨어	4GB+ RAM, x64 또는 ARM64	Apple Silicon 네이티브 지원
셸	Bash, Zsh, PowerShell, CMD	Windows는 CMD도 공식 지원
네트워크	인터넷 연결 필수, HTTPS 443	프록시, 커스텀 CA 지원
의존성	ripgrep 기본 포함, Node.js 불필요	npm 경로 사용 시에만 Node 18+

설치 방법 비교

다섯 가지 경로, 무엇을 선택할까

Native (권장)	curl / irm 원라이너, 자동 업데이트	개인, 팀 기본 선택
Homebrew	claude-code(stable), claude-code@latest	macOS 패키지 일원화
WinGet	winget install Anthropic.ClaudeCode	Windows 표준 관리
apt / dnf / apk	서명된 공식 저장소, stable과 latest 채널	리눅스 플릿 배포
npm (레거시)	@anthropic-ai/claude-code, Node 18+ 필요	기존 파이프라인 호환

Native
공식 권장

Auto-update
Native만 백그라운드

Signed
리눅스 저장소 GPG 서명

2 채널
stable / latest

Native Install / macOS, Linux, WSL

권장 설치, 원라이너로 완료

●●● INSTALL SCRIPT

```
# 설치 (macOS, Linux, WSL 공통)
curl -fsSL https://claude.ai/install.sh | bash

# 프로젝트에서 첫 실행
cd your-project
claude

# 처음 실행 시 브라우저 로그인 안내가 표시됩니다

# 특징
# Node.js 불필요, 단일 바이너리
# 백그라운드 자동 업데이트 기본 활성화
# 403, syntax error 발생 시 troubleshoot-install 문서 참조
```

1 line
설치 명령

No Node
런타임 의존성 없음

Auto
백그라운드 업데이트

install.sh
claude.ai 공식 스크립트

Native Install / Windows

PowerShell과 CMD를 구분

POWERSHELL / CMD

Windows PowerShell (프롬프트가 PS C:\)

irm https://claude.ai/install.ps1 | iex

:: Windows CMD (프롬프트가 C:\)

curl -fsSL https://claude.ai/install.cmd -o install.cmd

install.cmd && del install.cmd

판별 팁

'&&' is not a valid statement -> PowerShell

'irm' is not recognized -> CMD

GIT FOR WINDOWS

- 설치 권장, Bash 도구 활성화
- 없으면 PowerShell 도구로 대체
- WSL 설치라면 불필요
- 경로 지정:
CLAUDE_CODE_GIT_BASH_PATH

Homebrew 설치

stable과 latest, 두 개의 cask

HOME BREW

```
# stable 채널 (약 1주 지연, 큰 회귀 릴리스 제외)
```

```
brew install --cask claude-code
```

```
# latest 채널 (릴리스 즉시 반영)
```

```
brew install --cask claude-code@latest
```

```
# 업그레이드 (자동 업데이트 없음)
```

```
brew upgrade claude-code
```

```
brew upgrade claude-code@latest
```

```
# 디스크 정리 (이전 버전 보관 특성)
```

```
brew cleanup
```

2 casks

stable / @latest

~1주

stable 지연 폭

Manual

brew upgrade 필요

cleanup

구버전 정리 습관

WinGet 설치

Windows 표준 패키지 관리

● ● ● WINGET

설치

```
winget install Anthropic.ClaudeCode
```

업그레이드 (자동 업데이트 없음)

```
winget upgrade Anthropic.ClaudeCode
```

참고

실행 중에는 파일 잠금으로 업그레이드 실패 가능

CLAUDE_CODE_PACKAGE_MANAGER_AUTO_UPDATE=1 로

백그라운드 업그레이드 위임 옵션 제공

winget

Windows 표준 CLI

Manual

기본은 수동 업그레이드

Lock 주의

실행 중 업그레이드 실패

ENV=1

자동 업그레이드 위임

Linux 패키지 / apt

Debian, Ubuntu 서명 저장소

● ● ● APT (STABLE CHANNEL)

```
sudo install -d -m 0755 /etc/apt/keyrings
sudo curl -fsSL https://downloads.claude.ai/keys/claude-code.asc \
  -o /etc/apt/keyrings/claude-code.asc
echo "deb [signed-by=/etc/apt/keyrings/claude-code.asc] \
  https://downloads.claude.ai/claude-code/apt/stable stable main" \
  | sudo tee /etc/apt/sources.list.d/claude-code.list
sudo apt update && sudo apt install claude-code
```

GPG 지문 검증 (신뢰 전 필수)

```
gpg --show-keys /etc/apt/keyrings/claude-code.asc
# 31DD DE24 DDFA B679 F42D 7BD2 BAA9 29FF 1A7E CACE
```

Signed

GPG 서명 저장소

stable

기본 채널

latest

URL과 suite 교체

apt upgrade

시스템 절차로 갱신

Linux 패키지 / dnf, apk

Fedora, RHEL, Alpine

● ● ● DNF / APK

```
# Fedora / RHEL (dnf)
sudo tee /etc/yum.repos.d/claude-code.repo <<'EOF'
[claude-code]
name=Claude Code
baseurl=https://downloads.claude.ai/claude-code/rpm/stable
enabled=1
gpgcheck=1
EOF
sudo dnf install claude-code
# Alpine 준비물 (musl 계열)
apk add libgcc libstdc++ ripgrep
# settings.json: "USE_BUILTIN_RIPGREP": "0"
```

rpm/stable

dnf baseurl

gpgcheck=1

서명 검증 활성화

Alpine

libgcc, libstdc++ 필요

3.19+

Alpine 지원 버전

버전 지정과 채널 선택

특정 버전, stable 채널로 설치

PINNED INSTALL

```
# stable 채널로 설치 (설치 채널이 이후 업데이트 기본값)  
curl -fsSL https://claude.ai/install.sh | bash -s stable
```

```
# 특정 버전 설치  
curl -fsSL https://claude.ai/install.sh | bash -s 2.1.89
```

```
# Windows PowerShell에서 버전 지정  
& ([scriptblock]::Create((irm https://claude.ai/install.ps1))) 2.1.89
```

```
# 채널 개념
```

```
# latest: 릴리스 즉시    stable: 약 1주 지연, 회귀 제외
```

-s stable

채널 지정 인자

-s 2.1.89

버전 고정 인자

채널 승계

설치 채널이 업데이트 기준

2 채널

latest / stable

npm 설치 (레거시)

기존 파이프라인 호환 경로

● ● ● NPM (LEGACY PATH)

Node.js 18 LTS 이상 필요

node --version

글로벌 설치

npm install -g @anthropic-ai/claude-code

검증

claude --version

참고

글로벌 디렉토리 권한 문제 시 doctor가 해법 안내

신규 도입은 Native 경로 권장

Node 18+

npm 경로 요구

-g

글로벌 설치

doctor

권한 문제 진단

Legacy

신규는 Native 권장

Windows 실행 전략

Native, WSL2, WSL1 비교

Native Windows	추가 요구 없음, Git for Windows는 선택	샌드박스 미지원, Windows 도구체인
WSL 2	WSL 2 활성화 필요	샌드박스 지원, Linux 도구체인
WSL 1	WSL 1 활성화	샌드박스 미지원, WSL2 불가 시
셸 도구	Git Bash 있으면 Bash 도구	없으면 PowerShell 도구 사용

WSL2

샌드박스 지원 유일

Optional

Git for Windows

PS Tool

PowerShell 네이티브

프로젝트 위치

선택 기준 1순위

Docker 컨테이너

격리 실행과 이미지 표준화

● ● ● DOCKERFILE (EXAMPLE)

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y curl git ca-certificates
RUN curl -fsSL https://claude.ai/install.sh | bash
ENV PATH="/root/.local/bin:${PATH}"
WORKDIR /workspace

# 실행 예시 (키는 런타임 주입)
# docker run -it --rm \
#   -e ANTHROPIC_API_KEY \
#   -v $(pwd):/workspace claude-dev claude
```

Isolation

프로세스, FS 격리

Runtime -e

키는 이미지에 미포함

Volume

작업 디렉토리 마운트

Ch.3

엔터프라이즈 이미지 심화

Devcontainer 설정

팀 일관 개발 환경

```
.devcontainer/devcontainer.json

{
  "name": "claude-dev",
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu",
  "features": {},
  "postCreateCommand":
    "curl -fsSL https://claude.ai/install.sh | bash",
  "remoteEnv": {
    "ANTHROPIC_API_KEY": "${localEnv:ANTHROPIC_API_KEY}"
  }
}
```

postCreate

컨테이너 생성 시 설치

localEnv

호스트 키 전달

VS Code

Reopen in Container

Team

환경 일관성 확보

설치 검증

version과 doctor로 상태 확인

● ● ● VERIFY

버전 확인

claude --version

종합 자가 진단

claude doctor

설치 상태, 설정, 최근 업데이트 결과 점검

f 키를 누르면 발견된 문제를 Claude가 수정

즉시 수동 업데이트

claude update

--version

버전 출력

doctor

종합 진단

f

진단 후 자동 수정

update

즉시 갱신

자동 업데이트와 릴리스 채널

settings.json으로 채널과 하한 지정

settings.json

```
{
  "autoUpdatesChannel": "stable",
  "minimumVersion": "2.1.100",
  "env": {
    "DISABLE_AUTOUPDATER": "0"
  }
}
```

autoUpdatesChannel: latest(기본) 또는 stable
minimumVersion: 이 값 미만으로는 다운그레이드 금지
/config 의 Auto-update channel 항목으로도 변경 가능

latest

기본 채널

stable

약 1주 지연 채널

minimumVersion

다운그레이드 하한

/config

대화형 변경 경로

조직 차원 업데이트 통제

managed settings의 버전 정책

minimumVersion	업데이트가 이 값 미만 설치 거부	사용자 설정으로 무력화 불가
requiredMinimumVersion	미만 버전은 실행 자체 거부	managed 전용
requiredMaximumVersion	초과 버전 실행 거부, 업데이트 상한	검증된 범위 고정
DISABLE_AUTOUPDATER	백그라운드 확인만 중지	update 명령은 동작
DISABLE_UPDATES	수동 포함 모든 업데이트 차단	자체 배포 채널 조직용

제거 / Uninstall

설치 경로별 제거 방법

UNINSTALL

```
# Native (macOS, Linux, WSL)
rm -rf ~/.local/bin/claude ~/.local/share/claude

# Homebrew / WinGet
brew uninstall --cask claude-code
winget uninstall Anthropic.ClaudeCode

# apt / dnf
sudo apt remove claude-code
sudo dnf remove claude-code
# 사용자 데이터 정리 (선택, 세션 기록 포함 주의)
rm -rf ~/.claude ~/.claude.json
```

경로별

설치 방식 따라 제거

~/.claude

설정과 세션 저장소

주의

세션 기록 함께 삭제

purge

프로젝트 상태 정리 명령

설치 트러블슈팅

자주 만나는 오류와 해법

command not found

PATH에 설치 경로 누락

셸 재시작, PATH 확인

syntax error near '<'

프록시가 HTML 반환

프록시 예외, 직접 다운로드

403 / curl error

네트워크 차단, 지역 제한

방화벽 규칙, VPN 확인

'&&' not valid

CMD 명령을 PowerShell에서 실행

셸 구분 후 재시도

npm 권한 오류

글로벌 디렉토리 쓰기 불가

doctor 안내 수정, Native 전환

Part 3 Recap

Installation 핵심 정리

KEY TAKEAWAY

Native 원라이너가 새 표준입니다.

채널과 버전 하한, required 범위로 개인부터 조직 플릿까지 버전을 통제합니다.

핵심 정리

1. Native 권장, Node 불필요
2. Homebrew, WinGet, apt, dnf, apk
3. stable과 latest 두 채널
4. doctor로 진단, f로 자동 수정
5. required 버전 범위로 플릿 고정

다음 파트

1. Part 4 Authentication으로 이동
2. 구독 OAuth부터 Console까지
3. Bedrock, Vertex, Foundry 설정
4. apps gateway와 우선순위 6단계
5. CI용 setup-token

CHAPTER 01 / PART 04

Authentication

구독부터 Bedrock, Gateway까지 6가지 경로

1. Claude 구독 OAuth와 사용량 확인
2. Console API Key와 보안 수칙
3. Bedrock, Vertex AI, Microsoft Foundry
4. apps gateway, 우선순위, CI용 장기 토큰

인증 방법 한눈에

여섯 가지 경로 요약

구독 OAuth	claude 실행 후 브라우저 로그인	Pro, Max, Team, Enterprise
Claude Console	Console 계정, API 종량 청구	Claude Code 전용 롤 지원
Amazon Bedrock	AWS 자격증명, IAM 통제	엔터프라이즈 표준 경로
Google Vertex AI	GCP ADC 자격증명	GCP 표준화 조직
Microsoft Foundry	Azure 자격증명	Azure 표준화 조직
apps gateway	사내 IdP SSO, 게이트웨이 토큰	자체 호스팅 중앙 관문

Method 1 / Claude 구독 OAuth

가장 간단한 개인 경로

● ● ● FIRST LOGIN

\$ `claude`

브라우저가 열리며 로그인 진행

브라우저가 안 열리면

`c` 키: 로그인 URL 클립보드 복사

WSL2, SSH, 컨테이너처럼 콜백이 막힌 환경

브라우저에 표시된 코드를

'Paste code here if prompted'에 붙여넣기

\$ `/logout` # 로그아웃 후 재인증

ACCOUNT TYPES

- Pro, Max: 개인 구독
- Team, Enterprise: 조직 초대 계정
- Free 플랜은 사용 불가
- 자격증명은 OS 보안 저장소에 보관

구독 사용량 확인

/usage로 한도와 소비 내역 파악

● ● ● USAGE

\$ /usage

플랜 한도 대비 사용량, 리셋 시점 표시
무엇이 한도를 소모하는지 항목별 분해
스킬, 서브에이전트, MCP 서버별 기여 확인

\$ /cost # /usage 의 별칭

팁
한도 임박 시 haiku 모델로 전환해 소비 절감
/model haiku

/usage

한도와 소비 확인

분해 뷰

스킬, MCP별 기여

리셋

주기 시점 표시

/model

모델 전환 절감

Method 2 / Claude Console

API 종량 청구 조직 경로

CONSOLE PATH

Console 조직에 사용자를 초대하고 롤을 부여하면, 구성원이 Console 계정으로 로그인해 API 청구 기반으로 사용합니다.

STEP 1

조직 초대

Settings, Members에서
대량 초대 또는 SSO 연동.

STEP 2

롤 부여

Claude Code 롤은 Code
전용 키만, Developer 롤은
전체 키 생성.

STEP 3

사용자 로그인

설치 후 Console 자격증명
으로 로그인.

BILLING

종량 청구

조직 Console에 사용량 집
계, 예산 알림 설정 가능.

2 Roles

Claude Code / Developer

SSO

Console SSO 지원

Usage

조직 단위 집계

Invite

관리자 초대 필수

API Key 환경변수

셸과 컨테이너 환경 통합

ENV SETUP

```
# Bash / Zsh
export ANTHROPIC_API_KEY="sk-ant-..."

# PowerShell
$env:ANTHROPIC_API_KEY = "sk-ant-..."

# direnv (.envrc, 반드시 .gitignore에 추가)
export ANTHROPIC_API_KEY="sk-ant-..."

# Docker / Kubernetes
docker run -e ANTHROPIC_API_KEY ...
kubectl create secret generic claude-key \
  --from-literal=ANTHROPIC_API_KEY=sk-ant-...
```

X-API-Key

전송 헤더

1회 승인

대화형 최초 사용 시

.gitignore

.envrc 필수 등록

Secret

K8s는 시크릿으로

API Key 보안 수칙

유출 사고를 막는 네 가지 습관

RULE 1

코드에 금지

소스와 이미지에 키를 굵지
않기, 스캐너로 커밋 차단.

RULE 2

볼트 관리

Secrets Manager, Vault
에서 발급, apiKeyHelper
로 동적 주입.

RULE 3

최소 권한

Console의 Claude Code
롤로 키 용도 자체를 제한.

RULE 4

회전과 폐기

주기 회전, 퇴사와 유출 시
즉시 폐기 절차 문서화.

Helper

apiKeyHelper 동적 키

TTL

헬퍼 갱신 5분 기본

Scan

pre-commit 시크릿 스캔

Rotate

정기 회전 정책

Method 3 / AWS Bedrock

AWS 자격증명으로 추론 경로 전환

● ● ● BEDROCK ENV

```
# Bedrock 경로 활성화
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=ap-northeast-2

# 자격증명은 표준 AWS 체계 그대로
aws sso login --profile dev # 또는 IAM Role, 환경변수
export AWS_PROFILE=dev

# 실행
claude

# /status 로 활성 공급자와 모델 확인
```

USE_BEDROCK=1

경로 스위치

SSO/Role

표준 AWS 자격증명

Region

모델 제공 리전 확인

/status

활성 경로 검증

Bedrock 모델 해석과 지정

alias 매핑과 상위 모델 사용

opus / sonnet alias

Bedrock에서는 Opus 4.6, Sonnet 4.5로 해석

보수적 기본 매핑

상위 모델 사용

전체 모델명 또는 inference profile ARN 지정

claude --model로 전달

기본값 고정

ANTHROPIC_DEFAULT_OPUS_MODEL 등 환경변수

alias가 가리킬 버전 통제

소형 모델

ANTHROPIC_SMALL_FAST_MODEL로 배경 작업 지정

Haiku 계열 권장

리전 전략

모델별 제공 리전 상이, cross-region profile

지연과 가용성 균형

Bedrock IAM 정책

최소 권한 예시

● ● ● IAM POLICY (JSON)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ClaudeCodeBedrock",
    "Effect": "Allow",
    "Action": [
      "bedrock:InvokeModel",
      "bedrock:InvokeModelWithResponseStream"
    ],
    "Resource": "arn:aws:bedrock:*::foundation-model/anthropic.*"
  }]
}
```

2 Actions

Invoke + Stream

anthropic.*

리소스 범위 한정

SCP

조직 가드레일 병행

Ch.3

SSO 매핑 심화

Claude apps gateway

사내 SSO로 로그인하는 자체 호스팅 관문

SELF-HOSTED GATEWAY

게이트웨이가 IdP로 개발자를 인증하고, 추론을 설정된 클라우드로 라우팅합니다.

세션의 유일한 자격증명은 게이트웨이 토큰입니다.

STEP 1

/login SSO

개발자가 사내 IdP로 로그인

STEP 2

Gateway

그룹별 모델 접근, 지출 한도 적용

STEP 3

Provider

Bedrock, GCP, Foundry로 라우팅

STEP 4

Telemetry

OTLP로 사용량과 비용 집계

자격증명 우선순위 복습

충돌 시 무엇이 이기는가

0 Gateway 세션

apps gateway 로그인 상태면 아래 전부보다 우선

1 Cloud 스위치

USE_BEDROCK / VERTEX / FOUNDRY 환경변수

2-3 Token / Key

ANTHROPIC_AUTH_TOKEN, 그다음 ANTHROPIC_API_KEY

4 apiKeyHelper

스크립트 반환 동적 키, 볼트 연동

5-6 OAuth

CLAUDE_CODE_OAUTH_TOKEN, 마지막이 /login 구독

자격증명 저장 위치

플랫폼별 보관과 관리

macOS	암호화된 macOS Keychain	OS 보안 저장소
Linux	~/.claude/.credentials.json, 모드 0600	파일 권한 보호
Windows	%USERPROFILE%\W.claude\W.credentials.json	프로파일 ACL 상속
관리 명령	/login, /logout이 파일을 관리	직접 편집 비권장
커스텀 경로	CLAUDE_CONFIG_DIR로 위치 변경	Linux, Windows 해당

CI용 장기 토큰

setup-token으로 1년 OAuth 발급

● ● ● SETUP-TOKEN

```
# 브라우저 로그인이 불가능한 CI, 스크립트 환경용
claude setup-token
# OAuth 승인 후 토큰을 터미널에 출력 (저장 안 함)

# 사용
export CLAUDE_CODE_OAUTH_TOKEN=your-token
claude -p "analyze this repo"

# 특성
# 유효 1년, 구독 요금제 필요
# 추론 전용 스코프, Remote Control 불가
```

1년

토큰 유효 기간

Print only

출력만, 자동 저장 없음

Inference

추론 전용 스코프

CI Secret

파이프라인 시크릿 보관

/login, /logout, /status

세션 인증 관리 명령

● ● ● AUTH COMMANDS

\$ /status

활성 인증 방법, 공급자, 모델, 계정 표시

인증 문제 진단의 1순위 명령

\$ /logout

현재 자격증명 해제

\$ /login

재인증, gateway 환경에서는 SSO 흐름

\$ unset ANTHROPIC_API_KEY # 구독으로 복귀 시

/status

진단 1순위

/logout

자격증명 해제

/login

재인증, SSO 포함

unset

키 우선 문제 해결

Part 4 Recap

Authentication 핵심 정리

KEY TAKEAWAY

여섯 경로 중 하나를 고르고, 우선순위 규칙으로 충돌을 해석하며, /status로 검증합니다.

조직은 Bedrock 또는 gateway가 표준입니다.

핵심 정리

- 개인: 구독 OAuth가 기본
- 조직: Bedrock, gateway 표준
- alias 해석은 공급자별 상이
- 우선순위 6단계 + gateway 예외
- CI는 setup-token 장기 토큰

다음 파트

- Part 5 Quick Start로 이동
- REPL과 첫 프롬프트 요령
- 슬래시 명령 지도
- 권한 모드 실전 운용
- 세션 resume, branch, rewind

CHAPTER 01 / PART 05

Quick Start

첫 세션부터 세션 관리까지

1. claude 명령 기본형과 대화형 REPL
2. 첫 프롬프트 작성 요령과 슬래시 명령 지도
3. 디버깅, 기능 추가, 리팩토링 시나리오
4. 권한 모드 운용과 resume, branch, rewind

claude 명령 기본형

대화형과 헤드리스, 두 얼굴

COMMAND SHAPES

```
claude                # 대화형 REPL 시작
claude "질문 또는 작업"  # 초기 프롬프트와 함께 시작
claude -p "작업"        # 헤드리스, 결과만 출력 후 종료

claude --model opus     # 모델 지정 시작
claude --continue       # 최근 세션 이어서
claude --resume         # 세션 선택해 재개

cat data.csv | claude -p "요약" # 파이프 입력
claude --help           # 전체 플래그
```

REPL

기본 대화형

-p

헤드리스 모드

--model

시작 모델 지정

Pipe

stdin 입력 지원

대화형 REPL 입문

첫 세션의 화면 구성

● ● ● FIRST SESSION

```
$ cd my-project && claude
```

```
> 이 프로젝트가 뭘 하는지 설명해줘
```

```
# Claude가 Glob, Read로 구조 탐색하며 답변
```

```
> src/api의 에러 처리 방식을 알려줘
```

```
# 후속 질문은 이전 맥락을 그대로 활용
```

```
# 조작 키
```

```
# Esc: 즉시 중단    Esc Esc: 되감기
```

```
# Shift+Tab: 권한 모드 순환
```

```
# Ctrl+R: 명령 이력 검색
```

REPL BASICS

- 프롬프트에 자연어로 지시
- 도구 호출 과정이 실시간 표시
- 대화는 자동 저장, 재개 가능
- / 입력으로 명령 목록 필터

첫 프롬프트 작성 요령

공식 Best Practices 4원칙

DELEGATE, DON'T DICTATE

유능한 동료에게 위임하듯 맥락과 방향을 주고 세부 경로는 맡깁니다.

어떤 파일을 읽을지까지 지시할 필요는 없습니다.

TIP 1

구체적으로

관련 경로, 제약, 참고 패턴
을 처음부터 명시해 교정 횟
수 절감.

TIP 2

검증 기준 제공

테스트 케이스, 기대 출력 등
스스로 확인할 기준 제공.

TIP 3

탐색과 분리

복잡한 문제는 Plan 모드로
조사 먼저, 코딩은 그다음.

TIP 4

대화로 교정

완벽한 첫 프롬프트보다 중
간 개입과 반복 교정이 빠름.

경로 명시

src/payments/ 처럼

Verify

확인 가능한 기준

Plan 먼저

큰 변경 2단계

Esc

언제든 개입

명확한 vs 모호한 지시

같은 목표, 다른 결과

SPECIFICITY WINS

범위와 완료 기준이 담긴 지시가 1회 성공률을 결정합니다.

모호한 지시 (교정 비용 큼)

1. 로그인 버그 고쳐줘
2. 테스트 좀 추가해줘
3. 코드 정리해줘
4. 탐색 비용과 방향 이탈 위험 증가

명확한 지시 (1회 성공률 상승)

1. 만료 카드 결제 실패, src/payments/ 토큰 갱신부터 조사
2. validateEmail 구현, 3개 케이스 통과 후 테스트 실행
3. src/legacy만, 동작 변경 없이 이름 규칙 통일
4. 범위와 완료 기준이 함께 전달됨

슬래시 명령 지도

워크플로 단계별 핵심 명령

첫 세션	/init, /memory, /mcp, /agents, /permissions	프로젝트 셋업
작업 중	/plan, /model, /effort, /context, /compact, /btw	진행과 컨텍스트
병렬 실행	/agents, /tasks, /background, /batch	위임과 분산
배포 전	/diff, /code-review, /security-review	품질 게이트
세션 간	/clear, /resume, /branch, /teleport	전환과 재개
문제 시	/rewind, /doctor, /debug, /feedback	복구와 진단

/init 프로젝트 초기화

CLAUDE.md 자동 생성

INIT

```
$ claude
> /init
# 코드베이스를 분석해 CLAUDE.md 초안 생성
# 빌드, 테스트 명령과 컨벤션을 자동 발견
# AGENTS.md, .cursorrules 있으면 내용 반영
# 기존 CLAUDE.md가 있으면 개선안 제안

# 대화형 다단계 흐름 (선택)
$ CLAUDE_CODE_NEW_INIT=1 claude
> /init
# CLAUDE.md, skills, hooks 중 무엇을 만들지 질문
```

/init

초안 자동 생성

AGENTS.md

타 도구 설정 흡수

NEW_INIT=1

대화형 다단계

Part 8

CLAUDE.md 심화

/help, /status, /doctor

상태 확인 3종 세트

STATUS TRIO

> /help

사용 가능한 명령 전체와 설명

> /status

인증 방법, 공급자, 모델, 계정, 세션 정보

> /doctor

설치와 설정 종합 진단, 상태 아이콘 표시

f 키로 발견된 문제 자동 수정

/help

명령 카탈로그

/status

인증, 모델 확인

/doctor

종합 진단

f

자동 수정 트리거

/clear vs /compact vs /btw

컨텍스트를 다루는 세 가지 선택

CONTEXT TRIAGE

새 작업이면 clear, 같은 작업의 공간 확보면 compact, 흐름을 깨지 않을 결가지 질문이면 btw입니다.

/clear

새 대화 시작

컨텍스트 초기화, 이전 대화는
/resume에 보존. 별칭 /new.

/compact

요약해 지속

같은 작업을 이어가며 공간 확보.
초점 지시 전달 가능.

/btw

결가지 질문

대화 이력에 남지 않는 사이드 질문.
컨텍스트 오염 방지.

작업 전환

clear 선택 기준

공간 부족

compact 선택 기준

잡담, 확인

btw 선택 기준

/rewind

clear 이전 복귀 가능

/context 시각화

윈도우 사용량을 그리드로 확인

CONTEXT VIEW

> /context

- # 컨텍스트 사용량을 색상 그리드로 표시
- # System, CLAUDE.md, Auto memory, MCP,
- # 대화 이력, 도구 출력의 점유 비율
- # 과도한 항목에 대한 최적화 제안 표시

> /context all

- # 풀스크린 모드에서 항목별 상세 확장

> /mcp

- # 서버별 컨텍스트 비용 개별 확인

Grid

색상 점유 시각화

제안

최적화 힌트 포함

all

상세 확장 인자

/mcp

서버별 비용

시나리오 1 / 디버깅

실패 테스트를 통과시키기

SCENARIO WALKTHROUGH

\$ claude

> npm test가 실패해. checkout.test.ts의
만료 카드 케이스야. 원인 찾아서 고치고
테스트 통과까지 확인해줘.

관찰 포인트

- # 1. Bash로 테스트 실행, 실패 로그 수집
- # 2. 스택트레이스 파일을 Read
- # 3. Edit 승인 요청, diff 확인 후 수락
- # 4. 재실행 결과와 요약 보고

> 같은 패턴의 버그가 다른 곳에도 있는지 찾아줘

로그 먼저

실패 재현이 시작점

diff 확인

승인 전 변경 검토

재검증

통과까지가 완료

확장 질문

동일 패턴 수색

시나리오 2 / 기능 추가

Plan 모드로 2단계 진행

SCENARIO WALKTHROUGH

```
$ claude
# Shift+Tab 두 번: Plan 모드
> /users 목록 API에 커서 기반 페이지네이션 추가.
  기존 페이지 방식과의 호환도 고려해서 계획부터.

# Plan 산출물 검토 후
> 계획 2번 항목은 별도 PR로 빼고 나머지 진행해줘

# 구현 완료 후
> /diff          # 턴별, 파일별 변경 검토
> 커밋 메시지 작성해서 커밋까지 해줘
```

Shift+Tab x2

Plan 진입

계획 수정

승인 전 협상 가능

/diff

변경 검토 뷰어

Commit

메시지 자동 작성

시나리오 3 / 리팩토링

범위 고정과 회귀 방지

SCENARIO WALKTHROUGH

```
$ claude
> src/utils 안의 date 관련 함수들을 date-fns로 통일해줘. 동작 변경은 금지, 대상은 utils만.
  끝나면 lint와 테스트로 검증.

# 진행 중 개입 예시
# Esc 없이 타이핑 후 Enter:
# '가능하면 tree-shaking 친화적 개별 import로'
# 현재 도구 호출이 끝나는 즉시 반영됨

> /code-review --fix # 마무리 정리 리뷰
```

범위 고정

대상 디렉토리 명시

금지 조건

동작 변경 금지

달리는 중 개입

중단 없이 지시 추가

--fix

리뷰 결과 자동 적용

권한 모드 실전 운용

Shift+Tab 순환과 상황별 선택

MODE SWITCHING

모드는 세션 중 언제든지 전환됩니다.

큰 변경 전 Plan, 반복 작업엔 Accept Edits, 신뢰 저장소에선 Auto가 기본 감각입니다.

Shift+Tab

Plan

아키텍처 변경, 50+ 파일
, 의사결정

Shift+Tab

Default

일반 작업 기본값,
승인 흐름 학습

Shift+Tab

Accept Edits

소규모 반복 수정,
빠른 루프

Shift+Tab

Auto

신뢰 저장소,
분류기 안전 검사

Plan 모드 상세

탐색과 실행의 분리

PLAN MODE

소스 파일을 수정하지 않고 탐색과 설계만 수행합니다.

계획을 승인하면 실행으로 넘어가며, opusplan은 이 전환에 모델까지 바꿉니다.

ENTER

진입 방법

Shift+Tab 두 번 또는
/plan 명령으로 진입.

DURING

허용 작업

읽기, 검색, 웹 조회 가능.
소스 수정과 위험 명령 불가.

EXIT

계획 승인

산출 계획을 검토, 수정 요청
후 승인하면 실행 시작.

MODEL

opusplan

계획은 opus, 실행은
sonnet으로 자동 전환하는
alias.

50+ 파일

Plan 권장 규모

아키텍처

의사결정형 작업

협상 가능

승인 전 계획 수정

ultraplan

클라우드 확장판

Auto 모드 (Research Preview)

승인 피로 없이, 안전 검사와 함께

AUTO MODE

모든 행동을 백그라운드 안전 분류기가 평가합니다.

신뢰 저장소와 도메인을 조직이 설정하고, hard deny 규칙으로 무조건 차단선을 긋습니다.

HOW

동작 방식

행동별 위험을 분류기가 평가
, 안전하면 진행, 위험하면 확
인.

CONFIG

신뢰 설정

auto-mode 설정으로 신뢰
저장소, 버킷, 도메인 지정.

GUARD

Hard Deny

분류기와 무관하게 무조건
차단하는 규칙 계층 제공.

WHERE

가용 범위

Pro 플랜과 Sonnet 지원,
Bedrock, Vertex,
Foundry 확대.

Preview

리서치 프리뷰 상태

Classifier

백그라운드 안전 검사

hard deny

무조건 차단 규칙

Bedrock OK

클라우드 경로 지원

세션 관리 / resume, continue

대화를 이어가는 방법

SESSIONS

```
claude --continue      # 이 디렉토리의 최근 세션 재개
claude --resume        # 세션 목록에서 선택 재개
claude --from-pr 123    # PR을 만든 세션 찾아 재개
```

```
> /resume              # 세션 안에서 피커 열기
```

```
# 현재 워크트리 기준, 단축키로 범위 확장
```

```
> /clear 결제모듈-리팩토링 # 이전 대화에 이름 붙여 보관
```

```
# 세션은 ~/.claude/projects/ 에 JSONL로 저장
```

--continue

최근 세션 즉시

--resume

피커에서 선택

--from-pr

PR로 세션 추적

JSONL

로컬 저장 형식

세션 분기 / branch와 fork

다른 방향을 시험하는 두 방법

BRANCH VS FORK

branch는 내가 사본으로 갈아타 다른 방향을 시도하고, fork는 사본을 백그라운드 서브에이전트에게 맡깁니다.

/branch [이름]

1. 현 시점에서 대화 사본 생성
2. 나는 사본으로 전환해 작업
3. 원본은 /resume으로 복귀 가능
4. 실험적 방향 전환에 적합

/fork <지시>

1. 전체 맥락을 물려받은 서브에이전트 생성
2. 나는 원본에서 계속 진행
3. 완료되면 결과가 내 대화로 회수
4. 결가지 작업 위임에 적합

/rewind 체크포인트 복구

코드와 대화를 시점 단위로 롤백

REWIND

빠른 되감기

Esc Esc

최근 체크포인트로

> /rewind

체크포인트 목록에서 시점 선택

코드만 / 대화만 / 둘 다, 복구 범위 선택

/clear 이전 대화로도 복구 가능

한계

파일 변경만 복구, DB나 배포 등

외부 부작용은 대상 아님

Esc Esc

즉시 되감기

3 범위

코드, 대화, 둘 다

clear 복구

이전 대화 복원

Files only

외부 부작용 제외

/usage 비용과 한도

세션과 플랜 소비 확인

● ● ● USAGE

```
> /usage      # /cost 는 별칭
# 플랜 한도 대비 사용량과 리셋 시점
# 스킴, 서브에이전트, MCP 서버별 소비 분해

# 소비 절감 루틴
# /model haiku      가벼운 작업 전환
# /compact          이력 요약으로 토큰 절감
# /context          과소비 항목 식별

# 조직 집계는 OTel 텔레메트리로 (챕터 3)
```

/usage

한도와 분해 뷰

haiku

경량 전환 절감

/compact

이력 요약 절감

OTel

조직 단위 집계

Part 5 Recap

Quick Start 핵심 정리

KEY TAKEAWAY

구체적 지시와 검증 기준, 상황에 맞는 권한 모드, 그리고 resume와 rewind만 익혀도 일상 사용의 팔 할이 완성됩니다.

핵심 정리

1. 기본형: `claude, -p, --continue`
2. 프롬프트: 범위와 완료 기준 명시
3. 명령 지도: 단계별 6그룹
4. 모드: Plan, Default, Accept, Auto
5. 복구: `Esc Esc`와 `/rewind`

다음 파트

1. Part 6 Core Capabilities로 이동
2. 내장 도구 하나씩 심화
3. Git 워크플로와 gh 통합
4. 권한 규칙 문법
5. 샌드박스과 가드레일

CHAPTER 01 / PART 06

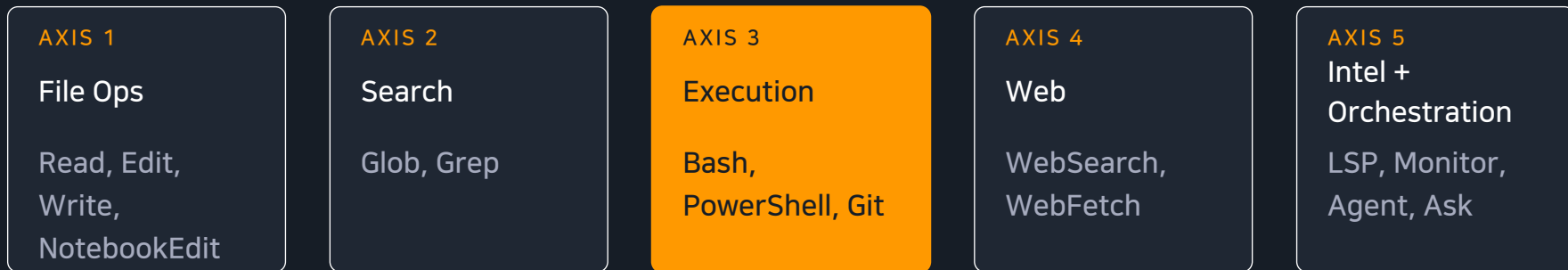
Core Capabilities

내장 도구를 손끝까지

1. 파일, 검색, 실행, 웹, 인텔리전스 5축 도구
2. LSP, Monitor, AskUserQuestion 등 신규 도구
3. Git 워크플로와 gh CLI 통합
4. 권한 규칙 문법과 샌드박스 가드레일

도구 카테고리 지도

이 파트의 학습 경로



도구 이름 문자열은 권한 규칙, 서브에이전트 tools 목록, Hook 매처에 그대로 사용됩니다

5 Axes

학습 경로

정확한 이름

규칙, 후에 재사용

승인 구분

부작용 도구만 확인

MCP

외부 도구 확장 축

Read / 파일 읽기와 멀티모달

텍스트, 이미지, 노트북까지

● ● ● READ PATTERNS

- > src/auth/session.ts 읽고 만료 로직 설명해줘
큰 파일은 필요한 라인 범위만 선택적으로 읽음
- > 이 스크린샷의 에러를 봐줘 (이미지 드래그, 붙여넣기)
PNG, JPG 등 이미지를 직접 이해
- > design.pdf의 3장 요구사항을 정리해줘
PDF 문서 읽기 지원
- # 특성: 승인 불필요, 접근은 작업 디렉토리 기준
- # 외부 경로는 --add-dir 또는 /add-dir로 확장

No approve

읽기는 승인 불필요

Image/PDF

멀티모달 입력

Range

라인 범위 선택 읽기

--add-dir

외부 경로 확장

Edit / 정밀 수정

diff 승인 기반의 안전한 변경

EDIT PATTERNS

> session.ts의 TTL 상수를 3600으로 바꾸고, 이 값을 참조하는 모든 곳을 함께 정리해줘

Edit 승인 화면

변경 전후 diff 표시

y 수락 / n 거절 / e 직접 수정

멀티 파일 수정

관련 파일들을 연속 Edit로 일괄 처리

각 편집 전 스냅샷 자동 저장 (rewind 가능)

diff 승인

변경 검토 후 수락

Multi-file

연속 편집 일괄 처리

Snapshot

편집 전 자동 저장

Accept Edits

모드로 자동화 가능

Write와 NotebookEdit

신규 파일과 Jupyter 셀

WRITE / NOTEBOOK

```
> tests/checkout.test.ts 파일을 새로 만들어 만료 카드 케이스 3개를 작성해줘
# Write: 신규 파일 생성, 승인 필요

> analysis.ipynb의 2번 셀을 pandas 2.x 문법으로 업데이트하고 실행 결과도 갱신해줘
# NotebookEdit: 셀 단위 수정

# 실행 가능한 파일 생성 시
# acceptEdits 모드에서도 확인 프롬프트 표시
```

Write

신규 파일 생성

NotebookEdit

Jupyter 셀 수정

승인 필요

두 도구 공통

실행 파일 주의

추가 확인 동작

Bash / 명령 실행

포그라운드와 백그라운드

● ● ● BASH PATTERNS

- > npm run build 실행하고 오류 있으면 고쳐줘
 - # 명령별 승인, allow 규칙으로 자주 쓰는 것 통과
- > 개발 서버를 백그라운드로 띄우고 로그에 에러가 보이면 알려줘
 - # 장시간 프로세스는 백그라운드 실행
 - # 이후 출력 감시는 Monitor 도구와 연계
- # 세션 특성
 - # 작업 디렉토리 유지, 타임아웃 존재
 - # 위험 명령은 deny 규칙으로 사전 차단

승인 필요

Bash 기본 정책

Background

장시간 프로세스

allow 규칙

반복 명령 통과

deny

위험 명령 차단

PowerShell 도구

Windows 네이티브 셸 지원

POWERSHELL

Git for Windows가 없는 네이티브 Windows

셸 명령이 PowerShell 도구로 실행됨

> Get-Process에서 메모리 상위 5개 보여줘

> 이 폴더의 오래된 로그 파일을 정리하는 PowerShell 스크립트를 만들어 실행해줘

Git Bash가 있는 환경에서의 선택적 사용

\$env:CLAUDE_CODE_USE_POWERSHELL_TOOL = 1

권한 규칙에서는 PowerShell(...) 로 매칭

Native Win

기본 셸 도구

Opt-in

Git Bash 병행 시 선택

PowerShell()

권한 규칙 표기

GPO 친화

Windows 운영 자동화

Grep과 Glob / 검색

내용 검색과 파일 패턴

SEARCH PATTERNS

> processPayment를 호출하는 곳을 전부 찾아줘

Grep: ripgrep 기반 내용 검색, 정규식 지원

> src 아래 *.test.ts 파일 목록 보여줘

Glob: 파일명 패턴 매칭

조합 패턴

> deprecated 주석이 달린 export 함수들을 찾아 사용처 개수와 함께 표로 정리해줘

Grep + Glob + Read를 스스로 조합

두 도구 모두 승인 불필요, 탐색이 빠른 이유

ripgrep

Grep 엔진

Regex

정규식 지원

No approve

검색 승인 불필요

조합

탐색 도구 자동 연계

LSP / 코드 인텔리전스

언어 서버 기반의 정적 이해

LANGUAGE SERVER TOOL

편집 직후 타입 오류와 경고를 확인하고, 정의로 이동하며, 참조를 찾습니다.

코드 인텔리전스 플러그인으로 언어를 확장합니다.

USE 1

편집 후 진단

Edit 직후 타입 오류를 확인해
즉시 재수정 루프.

USE 2

정의 이동

심볼의 정의 위치로 이동해
정확한 컨텍스트 확보.

USE 3

참조 찾기

변경 영향 범위를 참조 목록
으로 정량 파악.

SETUP

플러그인

언어별 code intelligence
플러그인 설치로 활성화.

Post-edit

수정 직후 진단

Definitions

정의, 참조 탐색

Plugins

언어별 확장

정확도

텍스트 검색 보완

WebSearch와 WebFetch

외부 지식을 세션 안으로

WEB PATTERNS

> 이 에러 메시지 최근 알려진 원인 검색해줘: ERR_OSSL_EVP_UNSUPPORTED

WebSearch: 웹 검색 결과 요약

> <https://code.claude.com/docs/en/hooks> 읽고 PreToolUse 입력 형식을 정리해줘

WebFetch: URL 콘텐츠 조회

통제

WebFetch(domain:docs.example.com) 형식으로

권한 규칙에서 도메인 단위 allow, deny

Search

최신 정보 조회

Fetch

문서 원문 반영

domain:

도메인 단위 통제

프록시

사내망 규칙 준수

Monitor 도구

출력을 지켜보다 반응하는 에이전트

● ● ● MONITOR

> npm run dev를 Monitor로 띄우고, 로그에 UnhandledPromiseRejection이 보이면 원인 파일을 찾아서 바로 고쳐줘

Monitor 동작

백그라운드 명령의 출력 라인을 실시간 수신

각 라인이 이벤트로 Claude에게 전달

WebSocket 연결의 수신 메시지도 이벤트화

활용: dev 서버 감시, 파일 변경 폴링,

장시간 배치의 진행 관찰과 자동 대응

Line events

출력 라인 단위 반응

WebSocket

수신 메시지 이벤트

승인 필요

실행 계열 정책

Loop 연계

관찰 후 자동 대응

Agent 도구 / 서브에이전트

챕터 2 미리보기

DELEGATION PRIMITIVE

Agent 도구는 독립 컨텍스트를 가진 서브에이전트를 생성합니다.
결가지 작업이 본 대화를 부풀리지 않고, 완료 시 요약만 돌아옵니다.

WHY

컨텍스트 격리

탐색성 작업의 대량 출력이 메인 대화를 오염하지 않음.

HOW

자동, 명시 호출

작업 성격에 따라 자동 위임,
또는 명시적으로 지정 호출.

SCALE

병렬 실행

독립 작업 여러 개를 동시에,
/batch는 워크트리 격리까지.

NEXT

챕터 2

맞춤 서브에이전트 정의와 5대
실전 패턴을 심화.

Isolated

독립 컨텍스트 윈도우

Summary

완료 시 요약 회수

Parallel

동시 실행 가능

Ch.2

심화 챕터

AskUserQuestion과 스케줄 도구

묻는 에이전트, 예약하는 에이전트

INTERACTIVE + SCHEDULED

모호하면 객관식으로 묻고, 반복 작업은 세션 안에서 예약합니다.

AskUserQuestion

1. 요구사항 모호 시 객관식 질문 제시
2. 잘못된 가정으로 달리는 것을 방지
3. 선택지가 곧 설계 옵션 정리
4. 승인 불필요, 흐름 자연 통합

CronCreate / List / Delete

1. 세션 내 반복, 일회성 프롬프트 예약
2. /loop로 폴링형 반복 실행
3. resume 시 미완료 예약 복원
4. 머신 독립 예약은 Routines (Part 9)

Git 워크플로 / 일상

상태 파악부터 스테이징까지

● ● ● GIT DAILY

> 지금 변경사항 요약해줘

git status, git diff를 읽고 자연어 요약

> 이 변경을 논리 단위로 나눠서 스테이징해줘

관련 파일끼리 묶어 git add 제안

> main과 충돌났어, 해결해줘

충돌 마커를 읽고 양쪽 의도를 병합

Claude는 git 상태를 항상 인지

현재 브랜치, 미커밋 변경, 최근 커밋 이력

status/diff

상태 자동 인지

논리 단위

스테이징 분할

Conflict

충돌 의도 병합

History

최근 커밋 참조

커밋 메시지와 PR 생성

이력을 읽고 컨벤션대로

COMMIT / PR

- > 설명적인 메시지로 커밋해줘
 - # 최근 커밋 스타일을 읽어 컨벤션 준수
 - # conventional commits 사용 저장소면 그 형식으로
- > 이 브랜치로 PR 만들어줘. 리뷰어가 봐야 할 변경 요점과 테스트 방법 포함해서.
 - # 브랜치 push, PR 본문 작성까지 일괄
- > /code-review --comment
 - # 리뷰 결과를 PR 인라인 코멘트로 게시

Convention

저장소 스타일 준수

PR 본문

요점과 테스트 방법

--comment

인라인 코멘트 게시

Ship loop

커밋부터 리뷰까지

gh CLI 통합

GitHub 작업을 대화로

● ● ● GH INTEGRATION

gh CLI가 설치되어 있으면 그대로 활용

> 나에게 할당된 열린 이슈 보여줘

gh issue list --assignee @me

> 123번 이슈 읽고 이 저장소에서 재현되는지 확인해줘

gh issue view 123 후 코드 탐색

> CI 실패한 최근 PR의 로그 분석해줘

gh run list, gh run view --log 조합

인증은 gh auth login 상태를 그대로 사용

gh

설치 시 자동 활용

Issues

조회와 재현 확인

CI logs

실패 원인 분석

gh auth

기존 인증 재사용

권한 규칙 문법

settings.json의 allow, ask, deny

.claude/settings.json

```
{
  "permissions": {
    "allow": ["Bash(npm test)",
              "Bash(npm run lint:*)", "Read(/src/**)"],
    "ask": ["Bash(git push:*)"],
    "deny": ["Read(/.env*)",
              "Bash(rm -rf:*)", "WebFetch"]
  }
}
```

Tool(pattern)

규칙 기본형

:*

접두 와일드카드

구체성 우선

충돌 시 해석 원칙

deny 최우선

차단이 항상 승리

도구별 권한 매트릭스

무엇이 승인을 요구하는가

Read, Glob, Grep

승인 불필요

탐색 속도의 원천

Edit, Write, NotebookEdit

승인 필요

diff 검토 후 수락

Bash, PowerShell, Monitor

승인 필요

allow 규칙으로 통과 가능

WebSearch, WebFetch

규칙 기반

도메인 단위 통제 권장

Agent, AskUserQuestion

승인 불필요

위임과 질문은 안전 동작

Sandboxed Bash

파일시스템과 네트워크 격리 실행

SANDBOXING

샌드박스 Bash는 명령을 격리 환경에서 실행합니다.

쓰기 가능한 경로와 나갈 수 있는 도메인을 정책으로 제한해, 더 높은 자율성을 안전하게 허용합니다.

FS

파일시스템 격리

정책이 허용한 경로만 쓰기 가능,
그 외는 읽기 전용.

NET

네트워크 격리

허용 도메인 목록 밖의
아웃바운드 차단.

WHERE

지원 플랫폼

macOS와 Linux, WSL2 지원,
네이티브 Windows 미지원.

ALT

격리 스펙트럼

샌드박스, devcontainer,
Docker, VM 중 위협 모델로
선택.

FS + Net

격리 두 축

WSL2

Windows 지원 경로

Policy

경로, 도메인 정책

Autonomy

안전한 자율성 확대

Safety 가드레일 종합

이 파트에서 배운 안전선

GUARD 1

권한 규칙

allow, ask, deny와 도구
패턴으로 사전 통제.

GUARD 2

모드 선택

Plan부터 Auto까지 위험
도에 맞춘 자율성 다이얼.

GUARD 3

체크포인트

편집 전 스냅샷과 rewind
로 사후 복구 보장.

GUARD 4

샌드박스

경로와 도메인 격리로 실행
반경 자체를 제한.

사전

규칙과 모드

사후

체크포인트 복구

반경

샌드박스 격리

감사

세션 기록, Hooks

Part 6 Recap

Core Capabilities 핵심 정리

KEY TAKEAWAY

다섯 축의 도구와 정확한 이름, 승인 정책을 알면 권한 설계와 자동화가 모두 쉬워집니다.

핵심 정리

1. 파일: Read, Edit, Write, Notebook
2. 검색: Grep, Glob은 무승인
3. 실행: Bash, PowerShell, Monitor
4. 지능: LSP 진단, 정의, 참조
5. Git: 커밋, PR, gh 통합

다음 파트

1. Part 7 Interfaces로 이동
2. VS Code, JetBrains 확장
3. Desktop 앱과 예약 작업
4. Web, Teleport, Remote Control
5. Slack과 Chrome 통합

CHAPTER 01 / PART 07

Interfaces

터미널 밖의 Claude Code

1. VS Code, Cursor, JetBrains 확장
2. Desktop 앱: 병렬 세션과 예약 작업
3. Web과 Teleport, Remote Control
4. Slack, Chrome 통합과 단축키

인터페이스 선택 가이드

원하는 것과 최적 표면

휴대폰에서 로컬 세션 이어가기

Remote Control

실행은 내 머신 유지

로컬 시작, 모바일에서 계속

Web / iOS + Teleport

클라우드 세션 이동

반복 일정으로 실행

Routines / Desktop 예약

머신 꺼져도 Routines

PR 리뷰, 이슈 분류 자동화

GitHub Actions / GitLab

CI 통합

Slack 버그 리포트를 PR로

Slack 통합

@Claude 멘션

라이브 웹앱 디버깅

Chrome 통합

콘솔, DOM 접근

VS Code 확장 / 설치

Cursor까지 동일 계열

● ● ● INSTALL

```
# 확장 마켓플레이스에서 'Claude Code' 검색
# Cmd+Shift+X (macOS) / Ctrl+Shift+X

# 설치 후 실행
# Cmd+Shift+P -> 'Claude Code'
# 'Open in New Tab' 선택

# Cursor도 같은 확장 사용
# vscode:extension/anthropic.claude-code
# cursor:extension/anthropic.claude-code
```

REQUIREMENTS

1. Claude 구독 또는 서드파티 공급자
2. CLI 설정과 CLAUDE.md 그대로 공유
3. MCP 서버 구성도 동일 적용
4. 터미널 세션과 자유롭게 병행

VS Code / 주요 기능

에디터 경험의 강점

FEATURE 1

인라인 Diff

제안 변경을 에디터 diff로 검토
, 부분 수락 가능.

FEATURE 2

@-mentions

@파일, @심볼로 컨텍스트를
정확히 지목해 전달.

FEATURE 3

Plan 검토

Plan 모드 산출물을 패널에서
구조적으로 검토.

FEATURE 4

대화 이력

세션 이력을 에디터 안에서
탐색하고 재개.

Diff

부분 수락 검토

@

컨텍스트 지목

Panel

플랜 구조 검토

Selection

선택 영역 자동 공유

JetBrains 플러그인

IntelliJ, PyCharm, WebStorm 계열

JETBRAINS PLUGIN

Marketplace에서 Claude Code 플러그인을 설치하고 IDE를 재시작합니다.
별도 설치된 CLI가 필요합니다.

SETUP

설치 요건

Marketplace 플러그인 +
별도 CLI 설치, 재시작.

DIFF

대화형 Diff

IDE diff 뷰어로 변경 검토와
수락.

CONTEXT

선택 공유

에디터 선택 영역을 컨텍스트
로 전달.

KEYS

단축키

IDE 키맵 체계에 통합,
커스터마이징 가능.

Marketplace

공식 배포 채널

CLI 필요

별도 설치 전제

IDE diff

네이티브 뷰어

All IDEs

IntelliJ 계열 전체

Desktop 앱

병렬 세션 운영의 본진

DESKTOP APP

IDE와 터미널 밖의 독립 앱입니다.

여러 세션을 git 격리로 나란히 돌리고, 시각적 diff와 앱 프리뷰, PR 모니터링까지 제공합니다.

PARALLEL

병렬 세션

세션마다 git 격리,
드래그 앤 드롭 패널 배치.

VISUAL

시각 검토

diff 검토, 앱 프리뷰, 통합
터미널과 파일 에디터.

DISPATCH

Dispatch

휴대폰에서 작업을 보내면
데스크톱 세션으로 생성.

CLOUD

클라우드 세션

로컬과 클라우드 세션을
한 화면에서 함께 운영.

mac/Win

Linux는 베타

/desktop

CLI 세션 이관 명령

Git 격리

세션별 워크트리

PR watch

PR 모니터링

예약 실행 두 경로

Desktop 예약 vs Routines

SCHEDULED WORK

로컬 파일이 필요하다면 Desktop 예약, 머신이 꺼져도 돌아야 하면 Routines입니다.

DESKTOP SCHEDULED TASKS

1. 내 머신에서 실행
2. 로컬 파일과 도구 직접 접근
3. 아침 코드리뷰, 야간 감사 등
4. 머신이 켜져 있어야 동작

ROUTINES (claude.ai)

1. Anthropic 관리 인프라 실행
2. 머신 꺼져도 스케줄 유지
3. API 호출과 GitHub 이벤트 트리거
4. /schedule로 CLI에서도 생성

Web / claude.ai/code

설치 없는 클라우드 세션

CLAUDE CODE ON THE WEB

브라우저에서 GitHub 저장소를 연결해 클라우드 세션을 시작합니다.

장시간 작업을 걸어두고 나중에 확인하거나, 여러 작업을 병렬로 돌립니다.

START

시작 방법

claude.ai/code 접속, 저장소 연결, 작업 제출.

ENV

실행 환경

Anthropic 샌드박스 VM, 셋업 스크립트와 네트워크 설정.

MOBILE

iOS 연속

iOS 앱에서 같은 세션 관리, 이동 중 진행 확인.

AUTOFIX

PR 자동 수정

/autofix-pr로 CI 실패와 리뷰 코멘트에 자동 대응.

No install

브라우저만으로

VM

샌드박스 격리

Parallel

다중 작업 병렬

PR 산출

결과는 PR로 회수

Teleport와 Remote Control

세션을 환경 사이로 옮기기

SESSION MOBILITY

```
# 웹, iOS에서 시작한 세션을 터미널로 가져오기
claude --teleport

# 목록에서 클라우드 세션 선택, 로컬로 이어서

# 터미널 세션을 클라우드로 보내기
claude --remote "이 작업 계속 진행해줘"

# 로컬 세션을 폰, 브라우저에서 조종
> /remote-control

# claude.ai/code 또는 모바일 앱에서 접속
# 실행은 내 머신, 조작만 원격
```

--teleport

클라우드에서 로컬로

--remote

로컬에서 클라우드로

/remote-control

원격 조종 개시

구독 필요

claude.ai 연동 기능

Slack 통합

팀 채팅에서 PR까지

CLAUDE IN SLACK

채널에서 @Claude를 멘션해 작업을 위임합니다.

버그 리포트 스레드가 그대로 컨텍스트가 되고, 결과는 PR로 돌아옵니다.

TRIGGER

@Claude 멘션

채널과 스레드에서 멘션
으로 작업 시작.

CONTEXT

스레드 컨텍스트

리포트 대화 내용이 작업
배경으로 전달.

OUTPUT

PR 회신

수정 결과를 PR 링크로
스레드에 회신.

OPS

운영 시나리오

버그 트리아지, 온콜 보조,
질문 응대 자동화.

@Claude

시작 트리거

Thread

맥락 자동 수집

PR

산출물 형태

Triage

대표 활용

Chrome 통합 (Beta)

라이브 웹앱을 직접 조작

CLAUDE IN CHROME

Chrome 브라우저에 연결해 실제 페이지를 조작합니다.

콘솔 로그를 읽고, 폼을 채우고, DOM을 검사하며 프론트엔드를 디버깅합니다.

DEBUG

콘솔 진단

콘솔 에러와 네트워크 요청
을 읽어 원인 추적.

TEST

동작 검증

구현한 UI를 실제로 클릭하고
입력해 확인.

AUTOMATE

폼 자동화

반복 입력과 데이터 추출 작업
수행.

SETUP

/chrome

확장 설치 후 /chrome으로
연결 설정.

Beta

베타 상태

Console

로그, 네트워크 접근

E2E 감작

구현 즉시 검증

/chrome

설정 명령

터미널 환경 최적화

CLI 사용감을 끌어올리는 설정

Shift+Enter 줄바꿈

terminal-config 절차로 키 매핑

멀티라인 프롬프트 필수

완료 알림 벨

터미널 벨 설정으로 종료 신호

장시간 작업 백그라운드화

Vim 모드

/config에서 편집 키 바인딩 전환

Vim 사용자 이질감 제거

풀스크린 렌더링

깜빡임 없는 렌더링, 마우스 지원

/focus 뷰 사용 가능

상태줄

statusline으로 컨텍스트, 비용, git 표시

세션 상황 상시 노출

키보드 단축키 종합

손에 익혀야 할 조작

Esc / Esc Esc

즉시 중단 / 체크포인트 되감기

가장 자주 쓰는 두 동작

Shift+Tab

권한 모드 순환

Plan 진입은 두 번

Ctrl+R

전 프로젝트 명령 이력 검색

과거 프롬프트 재사용

/ 입력

명령 자동완성 필터

명령 탐색 기본기

! 접두

셸 모드 직접 실행, 출력에 응답

빠른 셸 확인

c (로그인)

로그인 URL 클립보드 복사

브라우저 미개방 시

Part 7 Recap

Interfaces 핵심 정리

KEY TAKEAWAY

표면은 달라도 엔진과 설정은 하나입니다.

원하는 것에서 출발해 표면을 고르고, 세션은 teleport와 remote로 자유롭게 이동합니다.

핵심 정리

1. 에디터: diff, @멘션, 플랜 검토
2. Desktop: 병렬 세션, Dispatch
3. Web: 클라우드 세션, autofix-pr
4. 이동: teleport, remote, 원격조종
5. 확장: Slack, Chrome 통합

다음 파트

1. Part 8 CLAUDE.md & Memory
2. 두 메모리 시스템의 분업
3. 계층 구조와 로딩 순서
4. Rules 디렉토리와 경로 스코프
5. Auto memory 운용

CHAPTER 01 / PART 08

CLAUDE.md & Memory

세션을 넘어 지식을 잇는 두 시스템

- CLAUDE.md와 Auto memory의 분업
- 4계층 메모리와 로딩 순서
- Import 문법과 .claude/rules 디렉토리
- AGENTS.md 상호운용과 실전 예시

두 가지 메모리 시스템

내가 쓰는 것과 Claude가 쓰는 것

COMPLEMENTARY MEMORY

둘 다 매 세션 시작 시 로드됩니다.

지시와 규칙은 CLAUDE.md에, Claude가 발견한 학습은 Auto memory에 쌓입니다.

CLAUDE.md (사용자 작성)

1. 코딩 표준, 워크플로, 아키텍처 지시
2. 프로젝트, 사용자, 조직 스코프
3. 버전 관리로 팀과 공유
4. 지시가 반복되면 여기에 기록

AUTO MEMORY (Claude 작성)

1. 빌드 명령, 디버깅 인사이트 자동 축적
2. 저장소 단위, 워크트리 간 공유
3. 매 세션 첫 200줄 또는 25KB 로드
4. 교정할수록 알아서 똑똑해짐

메모리 계층 구조

넓은 스코프에서 좁은 스코프로

1 Managed

macOS /Library/Application Support/ClaudeCode, Linux /etc/claude-code 의 CLAUDE.md, 조직 정책

2 User

~/.claude/CLAUDE.md, 모든 프로젝트에 적용되는 개인 선호

3 Project

./CLAUDE.md 또는 ~/.claude/CLAUDE.md, 버전 관리로 팀 공유

4 Local

./CLAUDE.local.md, .gitignore 대상, 개인 프로젝트 노트

로딩 순서와 병합

디렉토리 트리를 걸어 올라가며

HOW CLAUDE.MD LOADS

```
# foo/bar/ 에서 claude 실행 시  
# 상위로 걸어 올라가며 발견한 파일을 모두 병합
```

/etc/claude-code/CLAUDE.md	(조직)
~/.claude/CLAUDE.md	(사용자)
foo/CLAUDE.md	(상위)
foo/bar/CLAUDE.md	(작업 디렉토리)
foo/bar/CLAUDE.local.md	(개인, 마지막)

```
# 하위 디렉토리의 CLAUDE.md는 그 폴더의  
# 파일을 읽을 때 온디맨드 로드  
# HTML 주석은 컨텍스트 주입 전에 제거됨
```

병합

덮어쓰기 아님

가까운 순 뒤

작업 위치가 마지막

On-demand

하위 폴더 파일

주석 제거

HTML 코멘트 무료

Project Memory 실전

./CLAUDE.md 표준 골격

./CLAUDE.md

MyService

Commands

- Build: npm run build
- Test: npm test (커밋 전 필수)
- Lint: npm run lint:fix

Conventions

- 들여쓰기 2칸, 세미콜론 사용
- API 핸들러는 src/api/handlers/ 에 위치
- 에러는 AppError 클래스로 래핑

Architecture

- 인증은 src/auth, 세션은 Redis 보관

200줄 이하

권장 상한

검증 가능

구체적 문장

헤더 구조

스캔 가능성

팀 공유

버전 관리 포함

Auto Memory 운용

Claude가 스스로 적는 학습 노트

MEMORY.md

작업 중 발견한 빌드 명령, 프로젝트 패턴, 사용자 교정 사항을 Claude가 저장소 단위로 기록합니다.

매 세션 앞부분이 자동 로드됩니다.

WRITE

자동 축적

교정과 발견이 자동 기록,
별도 작성 노력 불필요.

LOAD

로드 상한

첫 200줄 또는 25KB,
초과분은 필요 시 조회.

SCOPE

저장소 단위

워크트리 간 공유,
프로젝트별 독립.

CONTROL

관리

내용 확인과 정리 요청 가능,
서브에이전트도 자체 보유.

MEMORY.md

저장 파일

200줄/25KB

세션 로드 상한

Repo 단위

워크트리 공유

지시 아님

학습 기록 성격

/memory와 # 단축

세션 안에서 메모리 다루기

MEMORY COMMANDS

> /memory

로드된 메모리 파일 목록과 편집 진입

> # 커밋 전에는 반드시 npm test 실행

문장 앞 # 은 빠른 메모리 추가 단축

어느 파일에 넣을지 선택 프롬프트 표시

> /init

초안 생성 또는 기존 파일 개선안

반복해서 말하게 되는 지시는

그 순간 # 으로 바로 박제하는 습관

/memory

목록과 편집

#

빠른 추가 단축

파일 선택

스코프 지정 프롬프트

습관화

반복 지시 즉시 기록

Import 문법

@경로로 파일 모듈화

IMPORTS

See @README for overview, @package.json for scripts.

Git workflow

@docs/git-instructions.md

Personal (워크트리 공유용, 흠 참조)

@~/ .claude/my-project-instructions.md

규칙

상대 경로는 선언 파일 기준, 최대 4단계 재귀

백틱 안 @경로는 임포트되지 않음: `@README`

외부 경로 임포트는 최초 1회 승인 다이얼로그

@path

임포트 문법

4 hops

재귀 깊이 상한

백틱 예외

언급만 할 때

승인

외부 경로 1회 확인

.claude/rules 디렉토리

주제별 파일로 규칙 분리

● ● ● RULES LAYOUT

```
your-project/  
  .claude/  
    CLAUDE.md      # 본체 지시  
    rules/  
      code-style.md # 스타일 규칙  
      testing.md    # 테스트 컨벤션  
      security.md   # 보안 요구사항  
      frontend/     # 하위 폴더 재귀 탐색  
        components.md
```

```
# paths 없는 규칙은 세션 시작 시 로드  
# 심볼릭 링크 지원: 사내 표준 공유 가능
```

rules/
주제별 분리

재귀 탐색
하위 폴더 포함

Symlink
사내 표준 링크

~/.claude/rules
개인 전역 규칙

Path-scoped Rules

경로 매칭 시에만 로드되는 규칙

.claude/rules/api-design.md

```
---
paths:
  - "src/api/**/*.ts"
  - "lib/**/*.{ts,tsx}"
---

# API Development Rules
- 모든 엔드포인트는 입력 검증 필수
- 표준 에러 응답 형식 사용
- OpenAPI 주석 포함

# 매칭 파일을 읽을 때만 로드, 컨텍스트 절약
```

paths:

frontmatter 필드

Glob

중괄호 확장 지원

On-match

읽을 때만 로드

절약

컨텍스트 최적화

AGENTS.md 상호운용

다른 에이전트 도구와 지시 공유

● ● ● CLAUDE.md

@AGENTS.md

Claude Code

src/billing/ 아래 변경은 Plan 모드 사용

대안: 심볼릭 링크

ln -s AGENTS.md CLAUDE.md

Windows는 권한 문제로 @import 방식 권장

/init 은 AGENTS.md, .cursorrules,

.windsurfrules 를 읽어 초안에 반영

@AGENTS.md

임포트로 공유

+추가 지시

Claude 전용 부가

Symlink

내용 동일 시 대안

/init 함수

타 도구 설정 반영

작성 베스트 프랙티스

지켜지는 메모리의 조건

CONTEXT, NOT CONFIG

메모리는 강제 설정이 아니라 컨텍스트입니다.

구체적이고 간결하며 일관될수록 잘 지켜집니다.

SIZE

200줄 이하

길수록 준수율 하락, 넘치면
rules와 skills로 분리.

STRUCT

헤더와 볼릿

스캔 가능한 구조,
밀집 문단 지양.

SPECIFIC

검증 가능하게

2칸 들여쓰기처럼
확인 가능한 표현 사용.

CONSIST

모순 제거

상충 규칙은 임의 선택 유발,
주기적 정리.

짧게

준수율과 반비례 길이

구체적

모호함 제거

일관성

상충 규칙 청소

Hook

강제 필요 시 대안

안티 패턴

메모리를 망치는 습관

피해야 할 것

1. 소셜 같은 장문 서사
2. 프로젝트와 무관한 일반 상식
3. 비밀값과 자격증명 기록
4. 매 세션 불필요한 절차 문서 전체
5. 모순되는 규칙 방치

대신 이렇게

1. 명령과 규칙 중심의 간결한 목록
2. 이 저장소 고유의 사실만
3. 비밀은 볼트, 경로만 언급
4. 다단계 절차는 skill로 분리
5. 분기별 메모리 정리 루틴

실전 예시 / Python 프로젝트

FastAPI 서비스 표준안

./CLAUDE.md (PYTHON)

```
# Payments Service (FastAPI)

## Commands
- Setup: uv sync
- Test: uv run pytest -x (커밋 전 필수)
- Lint: uv run ruff check --fix
## Conventions
- 타입 힌트 필수, mypy strict 통과
- 라우터는 app/routers/, 스키마는 app/schemas/
- 외부 호출은 httpx.AsyncClient 주입 방식
## Don't
- requirements.txt 직접 수정 금지 (uv 사용)
```

uv

패키지 관리 명시

Don't 섹션

금지 사항 명문화

위치 규칙

라우터, 스키마 경로

주입 패턴

테스트 용이성 규칙

실전 예시 / 모노레포

루트와 패키지의 역할 분담

MONOREPO LAYOUT

```
repo/  
  CLAUDE.md           # 전역: 워크스페이스 명령, 공통 규칙  
  .claude/rules/      # paths 스코프 규칙  
  packages/  
    web/CLAUDE.md     # web 작업 시 온디맨드 로드  
    api/CLAUDE.md     # api 전용 컨벤션  
  
# 다른 팀 파일 제외 (settings.json)  
{ "claudeMdExcludes": ["packages/legacy/**"] }  
  
# 루트는 얇게, 상세는 패키지와 rules로
```

루트 얇게

전역 최소화

On-demand

패키지별 로드

Excludes

무관 팀 파일 제외

paths

rules 스코프 병행

Part 8 Recap

CLAUDE.md & Memory 핵심 정리

KEY TAKEAWAY

지시는 CLAUDE.md에, 학습은 Auto memory에 쌓입니다.

짧고 구체적으로 유지하고, 커지면 rules와 skills로 분리합니다.

핵심 정리

1. 두 시스템: 지시 vs 자동 학습
2. 4계층: Managed, User, Project, Local
3. # 단축과 /memory 편집
4. @import 4단계, rules 경로 스코프
5. AGENTS.md는 임포트로 공유

다음 파트

1. Part 9 Workflow Patterns
2. EPC와 TDD 패턴
3. code-review와 멀티 에이전트
4. Headless와 Routines 자동화
5. goal 지속 실행

CHAPTER 01 / PART 09

Workflow Patterns

반복 가능한 실무 패턴 10선

1. Explore-Plan-Code와 TDD
2. code-review, 멀티 에이전트, 비주얼 워크플로
3. Headless, 파이프라인, CI, Routines
4. goal 지속 실행과 비용, 컨텍스트 전략

Pattern 1 / Explore-Plan-Code

가장 기본이 되는 3단계 리듬

EPC RHYTHM

탐색으로 이해를 만들고, 계획으로 방향을 합의한 뒤, 코드로 실행합니다.

큰 변경일수록 단계 분리 효과가 큼니다.

STEP 1

Explore

관련 코드 읽기, 흐름 추적,
아직 코딩 금지

STEP 2

Plan

Plan 모드로 접근 설계, 계획 협상과
승인

STEP 3

Code

승인된 계획대로 구현, 테스트로 검증

EPC 실전 예시

OAuth 지원 추가 시나리오

● ● ● EPC IN ACTION

```
$ claude
# E: 탐색
> src/auth를 읽고 세션 처리 방식을 정리해줘.
  아직 코드는 만들지 마.

# P: 계획 (Shift+Tab 두 번)
> 이 구조에 OAuth 지원을 추가하는 계획을 세워줘
# 계획 검토, 항목 수정 요청, 승인

# C: 구현
> 승인한 계획대로 구현하고 테스트까지 실행해줘
```

아직 금지

탐색 단계 선언

협상

계획은 수정 가능

opusplan

단계별 모델 배분

검증 포함

구현 완료 기준

Pattern 2 / TDD Workflow

실패하는 테스트가 곧 명세

TDD LOOP

> validateCoupon 함수를 TDD로 만들자.

1) 먼저 실패하는 테스트 작성:

만료 쿠폰 거부, 중복 사용 거부, 정상 쿠폰 할인을 반환

2) 테스트 실행해서 실패 확인

3) 통과할 최소 구현 작성

4) 재실행으로 전부 통과 확인 후 리팩토링

기대 출력을 명세로 제공하는 것이 핵심

mock 데이터가 아닌 실제 실행 결과로 검증

Red 먼저

실패 확인 단계

명세 = 테스트

케이스가 요구사항

최소 구현

과설계 방지

Green 후 정리

리팩토링 순서

Pattern 3 / Code Review

/code-review 명령 계열

REVIEW COMMANDS

```
> /code-review          # 현재 diff 리뷰
> /code-review high     # 노력 수준 지정
> /code-review --fix     # 발견 사항 자동 수정
> /code-review --comment # PR 인라인 코멘트 게시
> /code-review ultra     # 클라우드 멀티에이전트 심층

> /simplify             # 버그 탐색 없이 정리만
> /security-review      # 보안 관점 읽기 전용 검사

# 머지 전 로컬 게이트로 습관화
```

--fix

결과 자동 적용

ultra

클라우드 심층 리뷰

/simplify

정리 전용 모드

머지 전

로컬 품질 게이트

Pattern 4 / Multi-Agent와 /batch

큰 변경을 병렬 단위로

PARALLEL SCALE

```
# 걸가지 위임
> /fork 이 변경의 문서 업데이트를 맡아줘

# 대규모 분할 실행
> /batch src/ 전체를 Solid에서 React로 마이그레이션
# 동작:
# 1. 코드베이스 조사, 5-30개 독립 단위로 분해
# 2. 계획 승인 후 단위별 서브에이전트 생성
# 3. 각자 격리된 git worktree에서 구현과 테스트
# 4. 단위별 PR 오픈

> /tasks # 백그라운드 작업 현황
```

/batch

5-30 단위 분해

Worktree

단위별 격리

PR each

리뷰 가능한 산출

Ch.2

설계 심화 챕터

Pattern 5 / Visual Workflow

스크린샷이 명세가 되는 흐름

● ● ● VISUAL LOOP

> (디자인 시안 이미지 붙여넣기)

이 시안대로 ProfileCard 컴포넌트를 구현해줘.

구현 후 스토리북 스크린샷과 시안을 비교해서 다른 부분을 스스로 찾아 고쳐줘.

프론트엔드 검증 강화

> /chrome 연결 후:

> localhost:3000/profile 열어서 콘솔 에러 확인, 버튼 클릭했을 때 모달 동작도 검증해줘

이미지 입력

시안이 곧 명세

자가 비교

결과와 시안 대조

Chrome

실동작 검증

UI 루프

구현, 확인, 수정

Pattern 6 / Headless 자동화

-p 플래그로 스크립트 편입

HEADLESS BASICS

기본형

```
claude -p "package.json에서 outdated 의존성 요약"
```

파이프와 리다이렉트

```
tail -200 app.log | claude -p "이상 징후 보고"
```

```
claude -p "이 저장소 의존성 분석" > deps-report.md
```

도구 제한으로 안전한 무인 실행

```
claude -p "보안 이슈 스캔" \
```

```
--allowed-tools "Read,Grep,Glob" \
```

```
--permission-mode plan
```

```
claude -p "간단 분류 작업" --model haiku # 비용 절감
```

-p

비대화 실행

allowed-tools

도구 화이트리스트

permission-mode

무인 안전선

haiku

배치 비용 절감

Pattern 7 / Pipeline과 JSON

구조화 출력으로 후처리

JSON OUTPUT

```
claude -p "현재 디렉토리 스택 분석" \  
  --output-format json | jq '.result'
```

```
# result 메시지에 포함되는 것  
# result: 최종 응답 텍스트  
# modelUsage: 실제 사용 모델  
# usage: 토큰 사용량, 비용 근거  
# session_id: 후속 --resume 연계
```

```
# 실시간 이벤트 스트림  
claude -p "..." --output-format stream-json  
# 이벤트 단위 파싱으로 진행 표시 구현
```

json

구조화 결과

stream-json

이벤트 스트림

jq

표준 후처리 도구

session_id

체인 실행 연계

Pattern 8 / CI 통합

GitHub Actions 최소 예시

● ● ● .github/workflows/claude-review.yml

```
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: curl -fsSL https://claude.ai/install.sh | bash
      - run: |
          claude -p "이 PR의 diff를 리뷰하고 위험을 보고" \
            --allowed-tools "Read,Grep,Glob,Bash(git diff:*)"
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

Actions

공식 액션도 제공

Secrets

키는 시크릿 주입

읽기 위주

CI 도구 제한

Ch.5

패턴 심화 챕터

Pattern 9 / Routines 예약 실행

머신이 꺼져도 도는 자동화

ROUTINES

```
> /schedule
# Routine 생성 흐름 시작
# 예: 평일 09:00, 열린 PR 리뷰 상태 요약
#     Slack #dev 채널로 보고

# Routines 특성
# Anthropic 관리 인프라에서 실행
# 스케줄 외 API 호출, GitHub 이벤트 트리거
# 웹과 Desktop 앱에서도 생성, 관리

# 세션 내 반복은 /loop (머신 필요)
> /loop 10m CI 상태 확인하고 실패하면 원인 요약
```

/schedule

Routine 생성

Managed

관리형 인프라 실행

이벤트

API, GitHub 트리거

/loop

세션 내 반복 폴링

Pattern 10 / /goal 지속 실행

조건이 참이 될 때까지

GOAL

> /goal 전체 테스트 스위트가 통과하고 lint 경고가 0이 될 때

동작

Claude가 턴을 넘겨도 목표 달성까지 계속 작업

각 턴 종료 시 조건 충족 여부 스스로 점검

> /goal # 현재 목표 확인

> /goal clear # 목표 조기 해제

Fable 5.1과 조합하면 장시간 자율 작업에 최적

/goal

완료 조건 선언

Cross-turn

턴 경계 지속

자가 점검

조건 충족 확인

Fable 5.1

장기 자율 조합

비용 관리 전략

모델, 컨텍스트, 캐시의 3축

COST LEVERS

비용은 모델 선택, 컨텍스트 크기, 캐시 적중의 함수입니다.

/usage 분해 뷰로 소비처를 찾고 세 레버를 조정합니다.

MODEL

작업별 모델

일상은 sonnet, 난제만
opus, 배치는 haiku로 배분.

CONTEXT

가볍게 유지

clear, compact, 경로 스코프
rules로 윈도우 최소화.

CACHE

캐시 친화

잘은 모델 전환과 세션 중
CLAUDE.md 수정은 캐시 미스
유발.

MONITOR

관측

/usage 분해와 OTeI로 팀 단위
소비 추적.

3 Levers

모델, 컨텍스트, 캐시

/usage

소비처 분해

Cache miss

모델 전환 비용

OTel

팀 단위 추적

컨텍스트 관리 전략

언제 비우고 언제 요약하는가

CONTEXT DISCIPLINE

작업 경계마다 정리하는 습관이 긴 세션의 품질을 지킵니다.

언제 무엇을

1. 작업 전환: /clear (+이름)
2. 같은 작업 지속: /compact 초점 지시
3. 결가지 질문: /btw
4. 탐색 위임: 서브에이전트로 격리
5. 상태 점검: /context 주기 확인

영속화 원칙

1. 대화 초반 지시는 소실 가능
2. 영속 규칙은 CLAUDE.md로
3. Compact Instructions 섹션 활용
4. 반복 절차는 skill로 승격
5. 경로 한정 지식은 rules로

토큰 효율 베스트 프랙티스

같은 결과를 더 적은 토큰으로

TIP 1

경로 지목

저장소 전체 탐색 대신 관련
경로를 직접 지목.

TIP 2

출력 형식 지정

표로, 다섯 줄로 등
출력 길이를 명시적 통제.

TIP 3

MCP 다이어트

안 쓰는 서버 해제, /mcp로
서버별 비용 점검.

TIP 4

메모리 다이어트

CLAUDE.md 200줄 유지,
rules 경로 스코프 분리.

지목

탐색 토큰 절감

형식

출력 토큰 통제

/mcp

서버 비용 가시화

200줄

메모리 상한 습관

자주 마주치는 안티 패턴

비효율의 신호들

안티 패턴

1. 한 세션에 서로 다른 작업 뒤섞기
2. 거대 파일 통째로 붙여넣기
3. 검증 기준 없는 막연한 지시
4. 실패한 방향을 수동 재타이핑으로 반복
5. 모든 작업을 opus로

교정

1. 작업 경계마다 /clear
2. 경로 지목과 라인 범위 활용
3. 테스트, 기대 출력 명세 동봉
4. /rewind로 시점 복귀 후 재지시
5. 난이도별 모델 배분

패턴 조합 예시

신규 기능 하나의 전체 여정

COMBINED FLOW

```
# 1. EPC + Plan: 기능 설계
> (Plan 모드) /users API 페이지네이션 계획

# 2. 병렬화: 독립 단위 위임
> /fork 문서와 예제 코드 업데이트 맡아줘

# 3. 품질 게이트
> /code-review --fix && /security-review

# 4. 배송
> 커밋하고 PR 만들어줘. /autofix-pr 로 CI 감시

# 5. 정리
> /usage 확인, /clear 다음작업
```

5 Steps

설계부터 정리까지

fork

결가지 병렬화

2 Reviews

품질, 보안 게이트

autofix

CI 사후 감시

Part 9 Recap

Workflow Patterns 핵심 정리

KEY TAKEAWAY

패턴은 조합입니다.

EPC를 축으로 리뷰와 병렬화, 자동화를 엮고, 비용과 컨텍스트 규율로 지속 가능하게 만듭니다.

핵심 정리

1. 기본 리듬: Explore, Plan, Code
2. 품질: TDD와 code-review 계열
3. 확장: fork, batch, Routines, goal
4. 규율: usage, clear, compact
5. 명세 동봉이 모든 패턴의 공통점

다음

1. Part 10 Recap & Labs
2. 챕터 요약과 체크리스트
3. FAQ와 학습 자료
4. 실습 랩 5종 안내
5. Q&A와 다음 챕터 예고

CHAPTER 01 / PART 10

Recap & Labs

챕터 정리와 실습 5종

1. Chapter 1 핵심 요약과 체크리스트
2. FAQ와 추가 학습 자료
3. 실습 랩 5종 안내
4. Q&A와 다음 챕터 예고

Chapter 1 핵심 요약

열 개 파트를 여섯 문장으로

IDENTITY

정체성

Agentic Loop로 저장소 단위 작업을 완수하는 Anthropic 1차 도구.

ARCH

아키텍처

Interface, Harness, Model, Provider 4계층과 도구, 체크포인트.

SETUP

설치와 인증

Native 원라이너 설치, 여섯 인증 경로와 우선순위 규칙.

USE

사용법

구체적 지시와 검증 기준, 슬래시 명령 지도, 권한 모드 4종.

MEMORY

메모리

CLAUDE.md 4계층과 Auto memory, rules 경로 스코프.

PATTERN

패턴

EPC와 TDD, 리뷰, 병렬화, Headless와 Routines 자동화.

학습 목표 체크리스트

시작할 때의 네 가지 목표 점검

SELF CHECK

아래 네 문장에 스스로 답할 수 있다면 챕터 1의 목표를 달성한 것입니다.

막히는 항목은 해당 파트로 돌아가 복습하세요.

CHECK 1

설명할 수 있다

Agentic Loop와 자동완성 도구의 차이를 설명. (Part 1, 2)

CHECK 2

설치했다

본인 환경에 설치와 인증 완료, /status 확인. (Part 3, 4)

CHECK 3

다뤄봤다

시나리오 셋 중 하나 이상을 직접 수행. (Part 5, 6)

CHECK 4

관리한다

CLAUDE.md 작성과 clear, compact 운용. (Part 8, 9)

4 Checks

목표 점검 항목

Part 참조

복습 경로 표기

Labs

미완 항목은 실습으로

Self

자가 평가 방식

FAQ / 자주 나오는 질문

현장에서 가장 많이 받은 여섯 가지

코드가 학습에 쓰이나요

상업 계정 기본 미사용, ZDR 옵션

Part 1 데이터 정책

구독과 API 중 무엇이 좋나요

개인은 구독, 조직은 Bedrock,
Console

Part 4 비교표

Windows에서 잘 되나요

Native 공식 지원, 샌드박스만 WSL2

Part 3 전략표

실수로 망가뜨리면요

체크포인트, Esc Esc, /rewind

Part 5 복구

팀 표준은 어떻게 만드나요

CLAUDE.md와 rules 버전 관리

Part 8 실전 예시

비용 통제가 되나요

/usage 분해, 모델 배분, OTel

Part 9 비용 전략

추가 학습 자료

이 챕터 다음에 읽을 것들

공식 문서	code.claude.com/docs/ko/overview	본 텍스트의 1차 출처
Best Practices	docs 내 best-practices 문서	패턴의 원전
GitHub 저장소	github.com/anthropics/claude-code	이슈, 릴리스 노트
Bedrock 안내	aws.amazon.com/bedrock	AWS 경로 심화
MCP 표준	modelcontextprotocol.io	챕터 4 연습
워크샵 코드	github.com/whchoi98/claude-code-workshop	본 과정 실습 자료

Lab 1 / 설치와 첫 세션

소요 15분

LAB 1 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

15분

예상 소요

doctor

진단 통과 목표

/status

인증 확인 목표

Lab 1

workshop-code/ch1

Lab 2 / CLAUDE.md 만들기

소요 15분

● ● ● LAB 2 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

15분

예상 소요

/init

초안 자동 생성

단축

빠른 규칙 추가

즉답 확인

성공 판정 기준

Lab 3 / 디버깅 실습

소요 20분

● ● ● LAB 3 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

20분

예상 소요

diff 읽기

승인 습관 형성

재검증

완료 기준 체크

되감기

복구 경험 필수

Lab 4 / Plan 모드 기능 추가

소요 25분

● ● ● LAB 4 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

25분

예상 소요

계획 협상

미션 1 목표

/diff

변경 검토 체크

커밋 위임

배송 루프 완성

Lab 5 / Headless 자동화

소요 15분

LAB 5 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

15분

예상 소요

4 Steps

패턴 4종 체험

jq

구조화 후처리

Ch.5 예고

자동화 심화 연결

Q&A

질문과 토론

DISCUSSION

설치, 인증, 팀 도입 전략 등 무엇이든 좋습니다.

시간 관계상 못 다룬 질문은 워크샵 저장소 이슈로 남겨주시면 답변드리겠습니다.

TOPIC 1

환경과 설치

사내망, 프록시, Windows 전략 관련 질문.

TOPIC 2

Bedrock 도입

리전, 모델 선택, IAM 설계 관련 질문.

TOPIC 3

팀 확산

표준화, 교육, 비용 거버넌스 관련 질문.

GitHub

후속 질문 채널

Issues

워크샵 저장소

LinkedIn

개별 연락 환영

10분

현장 질의 시간

Next / Chapter 2 예고

Subagents Deep Dive

CHAPTER 2 PREVIEW

오늘 여러 번 등장한 서브에이전트를 하루 종일 파고듭니다.

나만의 에이전트를 정의하고, 병렬 오케스트레이션으로 팀의 처리량을 바꿉니다.

NEXT 1

에이전트 정의

.claude/agents/ 마크다운
으로 역할, 도구, 모델 지정.

NEXT 2

오케스트레이션

자동 위임 원리와 명시 호출,
병렬 실행 설계.

NEXT 3

실전 패턴 5

리뷰어, 테스터, 마이그레이
터 등 검증된 구성.

NEXT 4

batch 내부

워크트리 격리와 단위 분해
전략의 내부 동작.

Ch.2

Subagents

5 Patterns

실전 구성

Hands-on

에이전트 직접 제작

To be continued

다음 세션에서

References

본 챕터 공식 문서 출처 (code.claude.com)

Overview / How it works

[docs/ko/overview, how-claude-code-works](#)

Part 1, 2

Setup / Authentication

[docs/ko/setup, authentication](#)

Part 3, 4

Model config

[docs/ko/model-config](#)

Part 1, 2, 4

Commands / Interactive

[docs/ko/commands, interactive-mode](#)

Part 5

Tools reference

[docs/ko/tools-reference](#)

Part 6

Memory / Best practices

[docs/ko/memory, best-practices](#)

Part 8, 9

Thank you!

Choi, WooHyung / Prin. Solutions Architect, AWS Korea

whchoi@amazon.com | linkedin.com/in/woohyungchoi

Workshop code: github.com/whchoi98/claude-code-workshop